



SAPIENZA
UNIVERSITÀ DI ROMA

Introduzione agli Algoritmi

UNIVERSITÀ DEGLI STUDI DI ROMA "LA SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E
STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
ANGELO MONTI

19 ottobre 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Indice

Prefazione	1
Introduzione al corso	5
 I Complessità di un algoritmo	 7
1 Notazione asintotica	9
1.1 O grande	9
1.2 Omega	10
1.3 Theta	10
2 Sommatorie	11
2.1 Sommatorie algebriche	11
2.2 Sommatorie geometriche	11
2.3 Sommatorie armoniche	11
Esercizi svolti	13
 II Ricerca e ordinamento	 17
3 Problema della ricerca	19
3.1 Ricerca sequenziale	19
3.2 Ricerca binaria	20
4 Programmi ricorsivi	23
4.1 Equazioni di ricorrenza	24
5 Problema dell'ordinamento	29
5.1 Selection sort	29
5.2 Insertion sort	29
5.3 Bubble sort	30
5.4 Mergesort	31
5.5 Quicksort	33
5.6 Heapsort	35
5.7 Counting Sort	38
5.8 Bucket sort	39
Esercizi svolti	41
 III Strutture dati	 43
6 Strutture dati fondamentali	45
6.1 Array	45
6.2 Liste	46
6.3 Pile	48

6.4	Code	49
6.5	Alberi	50
7	Dizionari	53
7.1	Tablelle Hash	53
7.2	Alberi binari di ricerca	55
7.3	Alberi di ricerca bilanciati	56
	Esercizi svolti	61

Introduzione al corso

L'obiettivo del corso "Introduzione agli algoritmi" è studiare alcuni algoritmi "classici" per risolvere problemi di base.

Un algoritmo è la specificazione di una sequenza finita di operazioni (dette anche istruzioni) che consente di risolvere tutti i quesiti di una stessa classe.

Per una migliore comprensione dell'argomento, nel corso esploreremo le strutture dati più adatte da usare nei problemi proposti, valuteremo l'efficienza di un'algoritmo calcolandone il suo costo computazionale e svilupperemo capacità di problem solving.

Tutti gli algoritmi sono analizzati e spiegati con il linguaggio di programmazione Python, in quanto molto vicino alla sintassi dello pseudocodice.

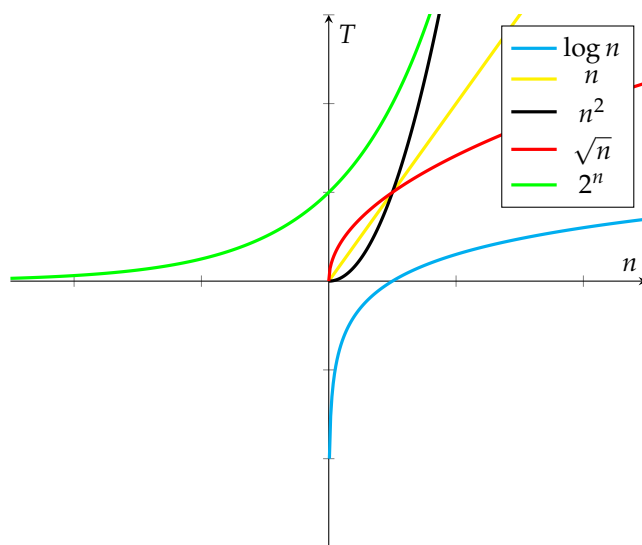
Parte I

Complessità di un algoritmo

Capitolo 1

Notazione asintotica

In matematica la notazione asintotica permette di confrontare il tasso di crescita (comportamento asintotico) di una funzione nei confronti di un'altra.



In informatica il calcolo asintotico è utilizzato per analizzare la complessità di un algoritmo. In particolare modo, per stimare quanto aumenta il tempo al crescere della dimensione n dell'input. Ma negli algoritmi non viene calcolata la costante del tempo, c'è bisogno di astrarre.

Esistono tre notazioni asintotiche:

- Notazione asintotica O . *O grande* è il limite superiore asintotico;
- Notazione asintotica Ω . *Omega* è il limite inferiore asintotico;
- Notazione asintotica Θ . *Theta* è il limite asintotico stretto.

1.1 O grande

L' O di una funzione, $O(g(n))$, è un'insieme di funzioni:

$$O(g(n)) = \{f(n) | \exists c, n_0, c > 0, n_0 \geq 0 \text{ t.c. } \forall n \geq n_0, f(n) \leq cg(n)\}$$

Ovvero rappresenta l'insieme di tutte le funzioni tali che devono essere dominate da $cg(n)$. Ad esempio $O(n)$ rappresenta tutte le funzioni che sono dominate da n .

Stando a quanto detto $5n + 38 \in O(n)$ perchè $5n + 38 \leq 5n + 38n = 43n \leq cn$. Infatti la funzione è dominata da cn con c almeno 43 ed $n \geq 1$. Si dice quindi che l'algoritmo assume la crescita di n perchè posso astrarre dalle costanti moltiplicative e additive.

Con qualche passaggio in più possiamo applicare lo stesso discorso per i polinomi di grado d . Ad esempio da $6n^3 - 2n + 5$ derivano $c_3 = 6, c_2 = 0, c_1 = 2, c_0 = 5$.

$$\sum_{i=0}^d c_i n^i = O(n^d)$$

Infatti $6n^3 - 2n + 5 \leq 6n^3 + 5 \leq 6n^3 + 5n^3 = 11n^3 \leq cn^3$ con $c \geq 11$ e $n \geq 1$.

Se invece voglio dimostrare che $\log_4 n = O(\log_2 n)$ dico che $\log_4 n \leq c \log_2 n$ ed effettuo un cambio di base: $\log_b n = \frac{1}{\log_a b} * \log_a n = c \log_a n$. Notiamo come nei logaritmi non è importante la base, a tal punto che si indica solamente $O(\log)$.

Come classe di tutte le costanti, infine, si prende sempre la classe 1. Ad esempio $50 \in O(1)$.

Se ho due funzioni e voglio sapere se $f(n) = O(g(n))$ posso calcolare il $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)}$. Affinchè sia vero che $f(n) = O(g(n))$ non devo ottenere $+\infty$, in tutti gli altri casi la funzione è dominata da $O(g(n))$.

1.2 Omega

L'Omega è sostanzialmente l'opposto dell'O grande. Intuitivamente con l'Omega dico che la complessità cresce almeno quanto ... ovvero non scende sotto a quel valore.

$$\Omega(g(n)) = \{f(n) | \exists c, n_0, c > 0, n_0 \geq 0 \text{ t.c. } \forall n \geq n_0, f(n) \geq cg(n)\}$$

Ovvero in $\Omega(g(n))$ troviamo tutte le funzioni che dominano la funzione $g(n)$. Anche con Ω posso astrarre dalle costanti moltiplicative e additive e applicare i metodi visti precedentemente.

1.3 Theta

Quando O e Omega coincidono otteniamo Theta. Diciamo quindi che la complessità è esattamente dell'ordine $\Theta(\dots)$ e per calcolarla è necessario che O e Ω coincidono, ovvero che il loro limite sia finito.

$$\Theta(g(n)) = \{f(n) | \exists c_1 > 0, c_2 > 0, n_0 \geq 0 \text{ t.c. } c_2 * g(n) \leq f(n) \leq c_1 * g(n), \forall n \geq n_0\}$$

$$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$$

Esempio 1.

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

- $O(n^2)$ perchè $\frac{n(n+1)}{2} \leq cn^2$
- $\Omega(n^2)$ perchè $\frac{n(n+1)}{2} \geq cn^2$
- $\Theta(n^2)$ perchè $O(n^2) = \Omega(n^2)$

In altro modo, direttamente con la sommatoria, vedo che:

- $\sum i = O(n^2)$ perchè $\sum_{i=1}^n i \leq \sum n = n^2 = cn^2$
- $\sum i = \Omega(n^2)$ perchè $\sum_{i=1}^n i = 1 + 2 + 3 + \dots + \frac{n}{2} + \dots + n \geq \frac{n}{2}(\frac{n}{2} + 1) + \dots + n \geq (\frac{n}{2})\frac{n}{2}$ volte
 $= \frac{n}{2} \frac{n}{2} = \frac{n^2}{4} = cn^2$
- $\Theta(n^2)$ perchè $O(n^2) = \Omega(n^2)$

Capitolo 2

Sommatorie

2.1 Sommatorie algebriche

Le sommatorie algebriche sono del tipo $\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$. A noi interessa il valore di grandezza quindi a volte è inutile calcolarle in maniera esplicita. Ad esempio:

$$\sum_{i=1}^n i^c = \Theta(n^{c+1}) \qquad \sum_{i=1}^n i^3 = \Theta(n^4)$$

2.2 Sommatorie geometriche

•

$$\sum_{i=0}^n c^i = \frac{c^{n+1} - 1}{c - 1}$$

$$\forall c \begin{cases} \Theta(c^n) & c > 1 \\ \Theta(n) & c = 1 \\ \Theta(1) & c < 1 \end{cases}$$

•

$$\sum_{i=1}^n ic^i \approx \Theta(nc^n) \qquad \sum_{i=1}^n i2^i \approx \Theta(n2^n)$$

•

$$\sum_{i=1}^n \log_2 i \approx \Theta(n \log n)$$

2.3 Sommatorie armoniche

•

$$\sum_{i=1}^n \frac{1}{i} \approx \int_1^n \frac{1}{x} dx = [\log_e x]_1^n = \Theta(\log n)$$

Esercizi svolti

Esercizio 1. Calcolare il costo computazionale del seguente algoritmo:

```
def es1(n):  
    if n < 0: n = -n  
    while n != 0:  
        if n % 2: return 1  
        n -= 2  
    return 0
```

In questo esercizio notiamo che se n è dispari la complessità è $O(1)$, se è pari $O(n)$. Ne deduciamo che $O(n)$ domina la complessità perchè l'algoritmo non farà mai peggio di $O(n)$.

Esercizio 2. Calcolare il costo computazionale del seguente algoritmo:

```
def es2(n):  
    n = abs(n)  
    x = z = 0  
    while x * x < n:  
        x += 1  
        z += 3x  
    return z
```

Il ciclo `while` viene eseguito $\sqrt{n}$ volte. La complessità è dell'ordine $\sqrt{n}$ e in particolar modo $O(\sqrt{n})$, $\Omega(\sqrt{n})$, quindi $\Theta(\sqrt{n})$.

Esercizio 3. Calcolare il costo computazionale del seguente algoritmo:

```
def es3(n):  
    n = abs(n)  
    x = z = 0  
    while n > 1:  
        r += 2  
        n = n // 3  
    return z
```

La complessità è $\Theta(\log(n))$ perchè il ciclo `while` viene eseguito $\frac{n}{3^i}$ volte, fino a quando $n = 1$. Quindi $\frac{n}{3^i} \approx 1$, $n \approx 3^i$, $\log_3 n \approx \log_3^i$.

Esercizio 4. Calcolare il costo computazionale del seguente algoritmo:

```
def es4(n):  
    n = abs(n)  
    t = x = 1  
    for i in range(n):  
        t = 3 * t  
    while t > x:  
        x += 2  
        t = t - 2  
    return z
```

La complessità del `for` è $\Theta(n)$. Dopo il `for` $t = 3^n$. Nel `while` t , dopo i primi passi, diventa $3^n - 2i$, mentre $x = 2i + 1$. Otteniamo così $3^n - 2i \approx 2i + 1 = 3^n \approx 4i$, con $i = \frac{3^n}{4} = \Theta(3^n)$.

Esercizio 5. Calcolare il costo computazionale del seguente algoritmo:

```
def es6(n):
    n=abs(n)
    u=t=s=1
    while u<=n:
        for j in range(t):
            s+=1
        u+=1
        t+=1
    return s
```

Il ciclo *while* viene eseguito $n + 1$ volte, mentre il *for* $\sum_{i=1}^n \Theta(i) = \Theta(\sum_{i=1}^n i) = \Theta(n^2)$ che è la complessità dell'algoritmo.

Esercizio 6. Calcolare il costo computazionale del seguente algoritmo:

```
def es7(n):
    n=abs(n)
    s=t=n
    p=0
    while s>=1:
        s=s//4
        p+=1
    while n>p:
        n-=p
        t+=5
    return t
```

Il primo ciclo *while* viene eseguito $\log_4(n)$ volte. Il secondo *while* viene eseguito $n - i \log(n) \approx 0$ ovvero $i = \frac{n}{\log(n)}$. Il programma ha complessità $\Theta(\frac{n}{\log(n)})$.

Esercizio 7. Calcolare il costo computazionale del seguente algoritmo:

```
def es8(n):
    n=abs(n)
    s=n
    p=2
    i=r=1
    while s>=1:
        s=s//5
        p+=2
    p=p*p
    while i*i*i>n:
        for j in range(p):
            r+=1
        i+=1
    return r
```

Il primo *while* viene eseguito $\Theta(\log(n))$ volte. Il secondo *while* viene eseguito $i^3 \approx n$ quindi $i = \sqrt[3]{n}$. Il *for* viene eseguito $\log^2(n)$ volte. Essendo il *for* dentro il *while* la complessità totale risulta essere $\Theta(\sqrt[3]{n} \log^2(n))$.

Esercizio 8. Calcolare il costo computazionale del seguente algoritmo:

```
def es9(n):
    tot=n
    if n<=100: return s
    j=512
    while j>=2:
        k=1
        while k*k<=n:
            k=2*k
```



```

    tot=tot+5
    j=j//2
    return t*t

```

Il primo *while* viene eseguito $\Theta(1)$ volte, perchè j è settato ad una costante: 512. Il secondo *while* viene eseguito $2^i * 2^i \approx n = 2^{2i} \approx n$. Il programma ha complessità totale $\Theta(\log(n))$.

Esercizio 9. Calcolare il costo computazionale del seguente algoritmo:

```

def es10(n):
    n=abs(n)
    p=2
    while n>=p:
        p=p*p
    return p

```

Il primo *while* viene eseguito $i = p^2, p^4, p^8, p^{2^i}$ quindi $2^{2^i} \approx n$ vuol dire che il programma ha complessità $\Theta(\log(\log(n)))$.

Parte II

Ricerca e ordinamento

Capitolo 3

Problema della ricerca

Nell'informatica esistono alcuni problemi particolarmente rilevanti. Uno tra questi è il problema della ricerca di un elemento in un insieme di dati.

Definiamo più formalmente tale problema descrivendone l'input e l'output:

- **Input:** una lista di numeri n numeri interi ed un valore v da ricercare.
- **Output:** un indice i tale che `lista[i]=v` oppure un particolare valore `Nome` se v non è presente nella lista.

3.1 Ricerca sequenziale

Un primo semplice algoritmo è basato sulla seguente idea. Ispezioniamo uno alla volta gli elementi del vettore, confrontando ciascun elemento con v . Se troviamo v restituiamo il suo indice, terminando l'algoritmo. Se v non è contenuto nel vettore, alla fine della scansione restituiamo `None`.

Da qui ne ricaviamo il seguente codice:

```
def ricercaSeq(A, v):  
    n=len(A)  
    for i in range(n):  
        if A[i]==v:  
            return i  
    return None
```

Questo algoritmo ha un costo di:

- $\Theta(n)$ nel caso peggiore (quando v non è presente nella lista);
- $\Theta(1)$ nel caso migliore (quando v è il primo elemento).

In queste situazioni diremo che il costo dell'algoritmo in generale è un $O(n)$, per evidenziare il concetto che l'algoritmo non può risultare peggiore di quella complessità.

Nei casi, come questo, in cui non sia possibile determinare un valore stretto per il costo computazionale, ed in cui il caso migliore e quello peggiore si discostano, è naturale domandarsi quale sia il costo computazionale dell'algoritmo nel **caso medio**.

Caso medio dell'algoritmo

Facciamo l'ipotesi che gli elementi del vettore siano distinti e che v possa apparire con uguale probabilità in qualunque posizione, ossia che:

$$P(\text{probabilità di trovare } v \text{ nell}'i\text{-esima posizione}) = \frac{1}{n}$$

Allora il numero medio di iterazioni del ciclo è dato da:

$$\sum_{i=1}^n i \frac{1}{n} = \frac{1}{n} \frac{n(n+1)}{2} = \frac{n+1}{2}$$

Un'ipotesi alternativa è la seguente: supponiamo che tutte le possibili $n!$ permutazioni della sequenza di numeri siano equiprobabili. Di queste, ve ne saranno un certo numero nelle quali v appare in prima posizione, un certo numero nelle quali v appare in seconda posizione, ecc. Il numero medio di iterazioni del ciclo sarà di conseguenza:

$$\sum_{i=1}^n i \frac{\text{numero di permutazioni in cui } v \text{ è in posizione } i}{\text{numero totale di permutazioni}}$$

Ora, il numero di permutazioni nelle quali v appare nella i -esima posizione è uguale al numero delle permutazioni di $n - 1$ elementi, cioè $(n - 1)!$ dato che fissiamo solo la posizione di uno degli n elementi.

$$\text{Numero medio di iterazioni} = \sum_{i=1}^n i \frac{(n-1)!}{n!} = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$$

Partendo da due diverse ipotesi di equiprobabilità abbiamo ottenuto lo stesso risultato: il costo computazionale nel caso medio è $\Theta(n)$.

3.2 Ricerca binaria

Con la ricerca binaria è possibile progettare un algoritmo molto più efficiente nel caso in cui la sequenza degli elementi sia ordinata.

L'idea è la seguente. Si ispeziona l'elemento centrale della sequenza, se esso è uguale al valore cercato ci si ferma. Se il valore cercato è più piccolo si prosegue nella sola metà inferiore della sequenza, altrimenti nella sola metà superiore.

Ad ogni iterazione si dimezza il numero degli elementi su cui proseguire l'indagine. Infatti dopo i iterazioni il numero di elementi su cui ricercare sarà $\frac{n}{2^i}$. Alla peggio ci si ferma quando per i si ha $\frac{n}{2^i} = 1$ vale a dire $2^i = n$ ovvero $i = \log n$.

Questo ci permette di capire perchè la ricerca binaria sia così efficiente: il numero di iterazioni cresce come $\log n$.

```
def RB(A, v):
    a, b = 0, len(A) - 1
    while a <= b:
        m = (a + b) // 2
        if A[m] == v:
            return m
        elif A[m] > v:
            b = m - 1
        else:
            a = m + 1
    return None
```

il costo computazionale è:

- $\Theta(\log n)$ nel caso peggiore (quando l'elemento non è presente nel vettore);
- $\Theta(1)$ nel caso migliore (quando l'elemento si trova a metà del vettore).

Poiché caso migliore e caso peggiore non hanno lo stesso costo, valutiamo il costo computazionale del caso medio.

Domandiamoci ora quante siano le posizioni $n(i)$ raggiungibili alla i -esima iterazione: con una iterazione si raggiunge la sola posizione $\frac{n}{2}$, cioè $n(1) = 2^0 = 1$; con due iterazioni si raggiungono le due posizioni $\frac{n}{4}$ e $3\frac{n}{4}$, cioè $n(2) = 2^1 = 2$; con tre iterazioni si raggiungono le quattro posizioni $\frac{n}{8}$, $3\frac{n}{8}$, $5\frac{n}{8}$, $7\frac{n}{8}$, cioè $n(3) = 2^2 = 4$.

In generale quindi, l'algoritmo di ricerca binaria esegue i iterazioni se e solo se v si trova in una delle $n(i) = 2^{i-1}$ posizioni raggiungibili con tale numero di iterazioni.

Ricordando che la probabilità che l'elemento si trovi su una delle $n(i)$ posizioni toccate dalla i -esima iterazione è $\frac{n(i)}{n}$ (ossia $\frac{2^{i-1}}{n}$), il numero medio di iterazioni è:

$$\text{Numero medio di iterazioni} = \sum_{i=1}^{\log n} i \frac{n(i)}{n} = \frac{1}{n} \sum_{i=1}^{\log n} i 2^{i-1}$$

e sapendo che $\sum_{i=1}^{\log n} i 2^{i-1} = (k-1)2k + 1$ otteniamo:

$$\frac{i}{n} ((\log n - 1) * 2^{\log n} + 1) = \log n - 1 + \frac{1}{n}$$

Ossia, il numero medio di iterazioni si discosta per meno di un confronto dal numero massimo di iterazioni!

Capitolo 4

Programmi ricorsivi

In questo capitolo ci occuperemo di calcolare la complessità di programmi ricorsivi, ovvero programmi che richiamano se stessi, dove un problema viene suddiviso in sotto-problemi simili a quello originale, ma più semplici.

Iniziamo guardando qualche esempio, per poi imparare a calcolare il costo computazionale di un algoritmo ricorsivo.

Esempio 2. Calcoliamo in maniera ricorsiva il fattoriale di un numero n .

```
def fatt(n):  
    if n == 0: return 1  
    return n*fatt(n-1)
```

Il costo computazionale di questo algoritmo dipende dai valori di n . Ovvero:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 0 \\ T(n-1) + \Theta(1) & \text{altrimenti} \end{cases}$$

La complessità totale di questo algoritmo risulta essere dell'ordine $\Theta(n)$.

Esempio 3. Ricerca di un elemento in un vettore ricorsivamente.

```
def ricercaR(x, A):  
    if A==[]: return False  
    if A[0] == x: return True  
    return ricercaR(x, A[1:])
```

Anche in questo caso il costo si divide in:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 0 \\ T(n-1) + \Theta(n) & \text{altrimenti} \end{cases}$$

Il secondo $\Theta(n)$ dipende dal fatto che nella ricorsione viene richiamata una copia della lista originale. Per tanto questa operazione di copia ha un costo che segue l'andamento di $\Theta(n)$. Si potrebbe risolvere questo problema considerando l'ultimo elemento della lista per poi eliminarlo quando viene passato come argomento della ricorsione. Il programma risultante sarebbe il seguente:

```
def ricercaR(x, A):  
    if A==[]: return False  
    if A[-1] == x: return True  
    A.pop()  
    return ricercaR(x, A)
```

In questo modo il programma ha costo $\Theta(n)$, invece che $\Theta(n^2)$.

Ci tengo a ricordare che questi algoritmi sono solo introduttivi per prendere dimestichezza con il concetto di ricorsione. Infatti uno stesso programma iterativo risulterebbe molto più efficiente.

Esempio 4. Quest'ultimo esempio riguarda il calcolo dell' i -esimo numero di Fibonacci.

```
def fibR(n):
    if n <= 1: return n
    return fibR(n-1) + fibR(n-2)
```

Questo è esattamente il tipo di programma ricorsivo che si vuole evitare. Scoprendo il costo computazionale dell'algoritmo capiremo il perché:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1 \\ T(n-1) + T(n-2) + \Theta(1) & \text{altrimenti} \end{cases}$$

Che si risolve in una complessità dell'ordine $\Theta(2^n)$.

Ma come facciamo a calcolare la complessità di questi algoritmi?

4.1 Equazioni di ricorrenza

In questa sezione vengono forniti alcuni metodi per risolvere le equazioni di ricorrenza, indispensabili per il calcolo del tempo computazionale degli algoritmi ricorsivi.

Esistono alcuni metodi utili per risolvere le equazioni di ricorrenza, che illustreremo:

- metodo iterativo;
- metodo dell'albero;
- metodo principale;
- metodo di sostituzione.

Trovare la funzione di costo ricorsiva è piuttosto immediato. Essa però deve essere riformulata in modo che non sia più ricorsiva, altrimenti il costo asintotico non può essere quantificato, e questa è la parte meno semplice. La riformulazione della funzione di costo ricorsiva si affronta impostando una equazione di ricorrenza, costituita dal passo ricorsivo e dal caso base.

Metodo iterativo

L'idea alla base di questo metodo è di sviluppare l'equazione di ricorrenza ed esprimerla come somma di termini dipendenti da n e dal caso base.

Introduciamo da subito qualche esempio per capire meglio:

Esempio 5.

- Ricerca sequenziale ricorsiva in una lista di n elementi.

```
def Ricerca_seq_ric(lista, v, i=0):
    if i == len(lista): return False
    if lista[i] == v: return True
    return Ricerca_seq_ric(lista, v, i+1)
```

Il corpo della funzione effettua un test, se tale test non è soddisfatto effettua una chiamata ricorsiva su un array di $(n-1)$ elementi. Dunque, nel corpo della funzione abbiamo un numero costante di operazioni elementari e una chiamata ricorsiva. Indicando con $T(n)$ il costo computazionale dell'algoritmo ricorsivo, possiamo esprimerlo mediante questa equazione di ricorrenza:

$$\begin{cases} T(n) = T(n-1) + \Theta(1) \\ T(1) = \Theta(1) \end{cases}$$

In generale, la parte generica dell'equazione di ricorrenza che definisce $T(n)$ deve essere sempre costituita dalla somma di almeno due addendi, di cui almeno uno contiene la parte ricorsiva, mentre uno rappresenta il costo computazionale di tutto ciò che viene eseguito al di fuori della chiamata ricorsiva.

Sviluppando ripetutamente la parte destra della prima equazione otteniamo:

$$T(n) = T(n-1) + \Theta(1) \quad T(n) = T(n-2) + \Theta(1) + \Theta(1)$$

$$T(n) = T(n-3) + \Theta(1) + \Theta(1) + \Theta(1)$$

Nell' i -esimo passo otterremo:

$$T(n) = T(n-i) + i\Theta(1) \quad n-i \approx 1 \quad n-1 \approx i$$

Dopo $n-1$ passi:

$$T(n) = T(1) + (n-1)\Theta(1) = n\Theta(1) = \Theta(n)$$

- Ricerca binaria ricorsiva in una lista di n elementi.

$$\begin{cases} T(n) = T(\frac{n}{2}) + \Theta(1) \\ T(1) = \Theta(1) \end{cases}$$

$$T(n) = T(\frac{n}{2}) + \Theta(1) \quad T(n) = T(\frac{n}{2^2}) + \Theta(1) + \Theta(1)$$

Dopo i passi:

$$T(n) = T(\frac{n}{2^i}) + i\Theta(1) \quad \frac{n}{2^i} \approx 1 \quad n \approx 2^i \quad \log(n) \approx i$$

Quindi dopo $\log(n)$ passi:

$$T(n) = T(1) + \log(n)\Theta(1) = \Theta(\log n)$$

- Risolvere la seguente equazione di ricorrenza:

$$\begin{cases} T(n) = 2T(\frac{n}{2}) + \Theta(1) \\ T(1) = \Theta(1) \end{cases}$$

$$T(n) = 2T(\frac{n}{2}) + \Theta(1) \quad T(n) = 2[2T(\frac{n}{2^2}) + \Theta(1)] + \Theta(1)$$

Dopo i passi:

$$T(n) = 2^i(\frac{n}{2^i}) + \sum_{j=0}^{i-1} 2^j\Theta(1) \quad \frac{n}{2^i} \approx 1 \quad i \approx \log(n)$$

$$T(n) = 2^{\log(n)}T(\frac{2}{2^{\log(n)}}) + \sum_{j=0}^{\log(n)-1} 2^j\Theta(1) = n\Theta(1) + n = \Theta(n) + \Theta(n) = \Theta(n)$$

Metodo dell'albero

La costruzione dell'albero di ricorrenza di un algoritmo ricorsivo è una tecnica per rappresentare graficamente lo sviluppo del costo computazionale dell'algoritmo, in modo da poterlo valutare con una certa facilità. Ogni nodo dell'albero è relativo alla soluzione di un problema di una certa dimensione. Esso riporta il costo della ricombinazione delle soluzioni parziali, ed ha tanti figli quanti sono i sotto-problemi in cui il problema relativo al nodo stesso è ricorsivamente scomposto.

Proponiamo un paio di esempi:

Esempio 6.

-

$$\begin{cases} T(n) = 2T(\frac{n}{2}) + \Theta(n^2) \\ T(1) = \Theta(1) \end{cases}$$

Si aggiunge un figlio per ogni chiamata ricorsiva effettuata (in questo caso due), tenendo traccia della dimensione del nuovo problema su cui si richiama l'algoritmo (in questo caso $n/2$ su entrambe le chiamate). Poi si itera il procedimento su ciascuno dei figli fino ad arrivare alle foglie, che corrispondono ai casi base e quindi non hanno figli. Una volta completato l'albero, il costo computazionale è dato dalla somma dei contributi di tutti i livelli.

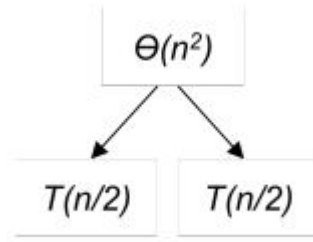


Figura 4.1: Rappresentazione dell'albero

Otterremo per tanto:

$$2^0 \Theta(n^2) \quad 2^1 \Theta\left(\left(\frac{n}{2}\right)^2\right) \quad 2^2 \Theta\left(\left(\frac{n}{2^2}\right)^2\right) \quad 2^i \Theta\left(\left(\frac{n}{2^i}\right)^2\right)$$

All' i -esima iterazione otterremo:

$$\sum_{i=0}^h 2^i \frac{n^2}{2^{2i}} = n^2 \sum_{i=0}^h \left(\frac{1}{2}\right)^i = n^2 \Theta(1) = \Theta(n^2)$$

•

$$\begin{cases} T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n) \\ T(1) = \Theta(1) \end{cases}$$

$$3^0 \Theta(n) \quad 3^1 \Theta\left(\frac{n}{2}\right) \quad 3^2 \Theta\left(\frac{n}{2^2}\right) \quad 3^i \Theta\left(\frac{n}{2^i}\right)$$

$$\sum_{k=0}^h 3^i \Theta\left(\frac{n}{2^i}\right) = \Theta(n) \sum_{i=0}^h \left(\frac{3}{2}\right)^i = \Theta(n) \Theta\left(\frac{3}{2}\right)^{\log_2(n)} = \Theta(n) \Theta\left(\frac{3^{\log_2(n)}}{2^{\log_2(n)}}\right) = \Theta(3^{\log_2(n)})$$

Metodo principale

Questo metodo è molto utile nella pratica e fornisce una soluzione meccanica per le equazioni di ricorrenza della forma:

$$\begin{cases} T(n) = aT\left(\frac{n}{b}\right) + \Theta(f(n)) \\ T(1) = \Theta(1) \end{cases}$$

Dove $a \geq 1$, $b > 1$ sono delle costanti ed $f(n)$ è una funzione asintoticamente positiva.

Enunciato del metodo principale

Il teorema principale può essere enunciato come segue. Dati $a \geq 1$, $b > 1$, una funzione asintoticamente positiva $f(n)$ ed un'equazione di ricorrenza della forma: $\begin{cases} T(n) = aT\left(\frac{n}{b}\right) + \Theta(f(n)) \\ T(1) = \Theta(1) \end{cases}$ vale che:

$$\begin{cases} f(n) = O(n^{\log_b a - \epsilon}) & T(n) = \Theta(n^{\log_b a}) \\ f(n) = \Theta(n^{\log_b a}) & T(n) = \Theta(n^{\log_b a} \log n) \\ f(n) = \Omega(n^{\log_b a + \epsilon}) & T(n) = \Theta(f(n)) \end{cases}$$

Con ϵ che rappresenta una costante > 0 . Nel primo caso si dice che f è polinomialmente dominata. Nel secondo hanno la stessa asintotica. Nel terzo f domina fortemente. E' necessario specificare la costante ϵ in quanto alcune funzioni della forma $aT\left(\frac{n}{b}\right) + \Theta(f(n))$ potrebbero non rientrare in nessuno dei tre casi.

Esempio 7.

•

$$\begin{cases} T(n) = 2T(\frac{n}{2}) + \Theta(\frac{n}{\log n}) \\ T(1) = \Theta(1) \end{cases}$$

Non rientra in nessun caso perchè confrontando

$$n^{\log_b a} = n^{\log_2 2} = n \quad \text{con} \quad f(n) = \frac{n}{\log n}$$

notiamo che nessun caso è verificato.

•

$$\begin{cases} T(n) = 3T(\frac{n}{2}) + \Theta(n) \\ T(1) = \Theta(1) \end{cases}$$

$$n^{\log_2 3} \quad f(n) = n \quad n = O(n^{\log_2 3 - \epsilon}) \quad \Theta(n^{\log_2 3})$$

•

$$\begin{cases} T(n) = T(\frac{n}{2}) + \Theta(1) \\ T(1) = \Theta(1) \end{cases}$$

$$n^{\log_2 1} = n^0 = 1 \quad f(n) = 1 \quad 1 = \Theta(1) \quad \Theta(1 \log n)$$

•

$$\begin{cases} T(n) = 2T(\frac{n}{2}) + \Theta(n) \\ T(1) = \Theta(1) \end{cases}$$

$$n^{\log_2 2} = n \quad f(n) = n \quad \Theta(n \log n)$$

Metodo della sostituzione

L'idea alla base di questo metodo è la seguente. Si ipotizza una soluzione per l'equazione di ricorrenza data; Si verifica se essa "funziona" procedendo per induzione. La difficoltà di questo metodo risiede nel fatto che si deve trovare la funzione più vicina alla vera soluzione.

Applichiamo questo metodo a qualche esempio:

•

$$\begin{cases} T(n) = T(n-1) + \Theta(1) \\ T(1) = \Theta(1) \end{cases}$$

Essendo l'equazione di ricorrenza dell'algoritmo ricorsivo di ricerca sequenziale sappiamo che vale $\Theta(n)$.

Ma in altra maniera seguiamo nel seguente modo:

$$\begin{cases} T(n) \leq T(n-1) + a \\ T(1) = b \end{cases} \rightarrow \exists c \quad T(n) \leq cn$$

Caso base: $T(1) = b \leq c * 1$ è verificato se prendo $c > b$;

Passo induttivo: $T(n) \leq T(n-1) + a = c(n-1) + a = cn + a - c \leq cn$ per $a - c \leq 0$ e quindi $a \leq c$.

Quindi abbiamo dimostrato che $T(n) = O(n)$.

Non resta che applicare lo stesso ragionamento per dimostrare:

$$\begin{cases} T(n) \leq T(n-1) + a \\ T(1) = b \end{cases} \rightarrow \exists c \quad T(n) \geq cn$$

Ottenendo infine $T(n) = \Theta(n)$.

•

$$\begin{cases} T(n) = T(n-1) + T(n-2) + \Theta(1) \\ T(0) = T(1) = \Theta(1) \end{cases}$$

Anche di questo algoritmo (calcolo dell' n -esimo numero di Fibonacci ricorsivo) conosciamo il costo computazionale. Ma proviamo a calcolarlo con il metodo dell'induzione:

$$T_1(n) \leq 2T(n-1) + \Theta(1) \quad \text{e} \quad T_2(n) \geq 2T(n-2) + \Theta(1)$$

$$T_2(n) \leq T(n) \leq T_1(n)$$

Risolvo T_1 ottenendo:

$$T_1 = 2T(n-1) + \Theta(1) = 2[2T(n-2) + \Theta(1)] + \Theta(1)$$

$$2^2 T(n-2) = 2^n \Theta(1) = \Theta(2^n)$$

Risolvendo T_2 invece otteniamo $\Theta(2^{\frac{n}{2}})$.

$$\Theta(2^{\frac{n}{2}}) \leq T(n) \leq \Theta(2^n)$$

Capitolo 5

Problema dell'ordinamento

In questo capitolo si affronta l'importante problema dell'ordinamento. Dapprima sono descritti degli algoritmi concettualmente semplici, che hanno costo computazionale quadratico nella dimensione dell'input. Poi ci si chiede se si possa sperare di trovare un algoritmo più efficiente; a tale scopo, si fornisce un teorema per cui nessun algoritmo di ordinamento che operi per confronti possa avere costo computazionale inferiore ad un ordine di $n \log n$.

Tutti gli algoritmi che vedremo hanno alcuni aspetti in comune. Si basano su due tipi fondamentali di operazioni: il confronto fra due elementi e lo scambio di due elementi. Inoltre, per semplicità, si suppone che gli n elementi da ordinare siano numeri interi e siano contenuti in un array i cui indici vanno da 0 a $n - 1$.

5.1 Selection sort

L'algoritmo *selection sort* (ordinamento per selezione) funziona nel seguente modo: cerca il minimo dell'intero array e lo mette in prima posizione; quindi cerca il nuovo minimo nell'array restante, cioè nelle posizioni dalla seconda all'ultima incluse, e lo mette in seconda posizione, e così via. In pratica, dopo l'iterazione i -esima, l'array risulterà suddiviso in due parti: quella di sinistra (di lunghezza i) contenente i primi i elementi della sequenza ordinata nel giusto ordine, e quella di destra (di lunghezza $n - i$) contenente gli altri elementi non ordinati. La formulazione dell'algoritmo è la seguente:

```
def Selection_Sort(A):
    n=len(A)
    for i in range(n-1):
        m = i
        for j in range(i + 1, n):
            if (A[j] < A[m]):
                m = j
        A[i], A[m]=A[m], A[i]
```

Il costo di questo algoritmo non dipende in alcun modo dalla distribuzione iniziale dei dati nell'array, cioè resta la stessa sia che l'array sia inizialmente ordinato o no. Essa è:

$$\sum_{i=0}^{n-2} n - 1 - i \quad j = n - 1 - i \quad \sum_{j=n-1}^1 j = \Theta(n^2)$$

5.2 Insertion sort

L'algoritmo *insertion sort* (ordinamento per inserzione) si effettua "estraendo" l'elemento così da liberare la sua posizione corrente, spostando poi verso destra tutti gli elementi alla sua sinistra (già ordinati) che sono maggiori di esso ed, infine, inserendo l'elemento nella posizione che si è liberata. Alla fine del procedimento l'array risulterà ordinato. La formulazione dell'algoritmo è la seguente:

```
def Insertion_Sort(A):
    for j in range(1, len(A)):
```

```

x = A[j]
i = j - 1
while ((i >= 0) and (A[i] > x)):
    A[i+1] = A[i]
    i = i - 1
A[i+1] = x

```

Valutiamo il costo computazionale dell'algoritmo distinguendo due casi:

Caso ottimo

Esso si verifica quando il numero di spostamenti da effettuare è minimo e cioè quando l'array è già ordinato. In tal caso la condizione del while non è mai verificata e quindi:

$$\sum_{i=0}^{n-1} \Theta(1) = \Theta(n)$$

Caso pessimo

Esso si verifica quando il numero di spostamenti da effettuare è massimo e cioè quando l'array in partenza è ordinato in maniera decrescente. Il costo è:

$$\sum_{i=0}^{n-1} i = \Theta(n^2)$$

5.3 Bubble sort

L'algoritmo *bubble sort* (ordinamento a bolle) ha un funzionamento molto semplice: l'algoritmo ispeziona una dopo l'altra ogni coppia di elementi adiacenti e, se l'ordine dei due elementi non è quello giusto, essi vengono scambiati. L'algoritmo prosegue fino a che non vi sono più coppie di elementi adiacenti nell'ordine sbagliato. La formulazione dell'algoritmo è la seguente:

```

def Bubble_Sort(A):
    n=len(A)
    for i in range(n-1):
        for j in range(n-i-1):
            if (A[j] > A[j+1]):
                A[j],A[j+1]=A[j+1],A[j]

```

Con questo procedimento il massimo valore viene portato alla fine del vettore, producendo una complessità dell'ordine:

$$\sum_{i=0}^{n-1} i = \Theta(n^2)$$

La complessità del problema dell'ordinamento

Abbiamo visto che i tre algoritmi di ordinamento precedenti hanno tutti un costo computazionale asintotico che cresce come $\Theta(n^2)$. Una domanda sorge spontanea: si può fare di meglio? E se sì, quanto meglio si può fare? In altre parole, come si fa a stabilire un limite di costo computazionale al di sotto del quale nessun algoritmo di ordinamento basato su confronti fra coppie di elementi possa andare?

Esiste uno strumento adatto allo scopo, l'albero di decisione. Esso permette di rappresentare tutte le strade che la computazione di uno specifico algoritmo può intraprendere, sulla base dei possibili esiti dei test previsti dall'algoritmo stesso. Un albero di questo tipo è formato dalle seguenti caratteristiche:

- è un albero binario che rappresenta tutti i possibili confronti che vengono effettuati dall'algoritmo; ogni nodo interno ha esattamente due figli;

- ogni nodo interno rappresenta un singolo confronto, ed i due figli del nodo sono relativi ai due possibili esiti di tale confronto;
- ogni foglia rappresenta una possibile soluzione del problema, la quale è una specifica permutazione della sequenza in ingresso.

Eseguire l'algoritmo corrisponde a scendere dalla radice dell'albero alla foglia che contiene la permutazione che costituisce la soluzione. Dunque, la lunghezza del percorso più lungo dalla radice ad una foglia (ossia l'altezza dell'albero binario) rappresenta il numero di confronti che l'algoritmo deve effettuare nel caso peggiore.

Ora, dato che la sequenza di ingresso può avere una qualunque delle sue permutazioni come soluzione, l'albero di decisione deve contenere nelle foglie tutte le permutazioni della sequenza in ingresso, che sono $n!$. D'altra parte, un albero binario di altezza h non può contenere più di 2^h foglie, quindi l'altezza h dell'albero di decisione di qualunque algoritmo di ordinamento basato su confronti deve essere tale per cui:

$$2^h \leq n! \quad h \leq \log n!$$

Da cui ricaviamo:

$$\log n! \leq \log\left(\frac{n}{2}\right) \frac{n}{2} = \frac{n}{2} \log \frac{n}{2} = \Theta(n \log n)$$

Teorema. Il costo computazionale di qualunque algoritmo di ordinamento basato su confronti è $\Omega(n \log n)$.

5.4 Mergesort

L'algoritmo *mergesort* (ordinamento per fusione) è un algoritmo ricorsivo che adotta una tecnica algoritmica detta **divide et impera**. Essa può essere descritta come segue:

- **divide:** la sequenza di n elementi viene divisa in due sotto-sequenze di $n/2$ elementi ciascuna;
- **impera:** le due sotto-sequenze di $n/2$ elementi vengono ordinate ricorsivamente;
- **passo base:** la ricorsione termina quando la sotto-sequenza è costituita di un solo elemento, per cui è già ordinata;
- **combina:** le due sotto-sequenze – ormai ordinate – di $n/2$ elementi ciascuna vengono “fuse” in un'unica sequenza ordinata di n elementi.

Lo pseudocodice dell'algoritmo mergesort è il seguente:

```
def Merge_sort (A, indice_primo, indice_ultimo):
    if (indice_primo < indice_ultimo):
        indice_medio = (indice_primo+indice_ultimo)//2
        Merge_sort (A, indice_primo, indice_medio)
        Merge_sort (A, indice_medio + 1, indice_ultimo)
    Fondi (A, indice_primo, indice_medio, indice_ultimo)
```

Vediamone il funzionamento su un problema costituito di 8 elementi, dando per scontato (solo per il momento) il funzionamento della funzione Fondi:

Si osservi che le caselle gialle del disegno corrispondono alle operazioni di suddivisione dell'array in sottoarray, cosa che avviene solo tramite le varie chiamate ricorsive. Le caselle azzurre del disegno invece indicano i casi base e quelle verdi corrispondono alle chiamate della funzione Fondi, le quali restituiscono porzioni di array ordinate sempre più grandi.

Illustriamo ora il funzionamento della fase di fusione:

```
def Fondi (A; indice_primo, indice_medio, indice_ultimo):
    i, j = indice_primo, indice_medio+1
    B=[]
    while ((i <= indice_medio) and (j <= indice_ultimo)):
        if (A[i] <= A[j]):
            B.append(A[i])
            i += 1
        else:
```

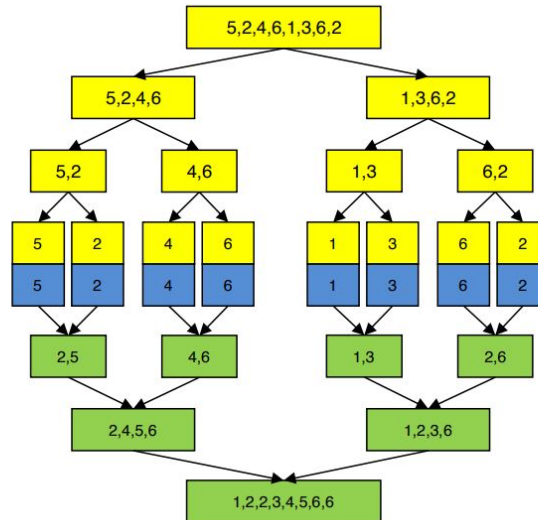


Figura 5.1: Procedimento del Merge sort

```

        B.append(A[j])
        j += 1
    while (i <= indice_medio)
        B.append(A[i])
        i += 1
    while (j <= indice_ultimo)
        B.append(A[j])
        j += 1
    for i in range(len(B)):
        A[indice_primo+i] = B[i]

```

Essa sfrutta il fatto che le due sotto-sequenze sono ordinate, cosa che rende facile sfruttare un ciclo che scorre partendo dall'inizio delle due sotto-sequenze, confronta i primi elementi alla ricerca del valore minimo da inserire nel vettore. In base al valore trovato (se si trova nella prima sequenza o nella seconda), incrementa l'indice della sotto-sequenza e ripete l'operazione. Alla fine di ciò i due cicli aggiuntivi controllano che non ci siano altri elementi da aggiungere, in caso contrario salva anche quelli nel vettore. In questo modo abbiamo creato un "vettore copia" che ci permette di salvare la sequenza ordinata.

Costo del Mergesort

Analizziamo il costo dell'operazione di fusione e poi quella dell'algoritmo mergesort.

Nella fusione si effettuano un ciclo while, nel quale per ogni elemento di ciascuna delle due sotto-sequenze si compie un numero costante di operazioni. Il costo computazionale del while è individuato dal numero di iterazioni: poiché si scorre un array di dimensione $\frac{n}{2}$ quindi $\Theta(n)$.

L'equazione di ricorrenza del mergesort è di conseguenza:

$$\begin{cases} T(n) = 2T(\frac{n}{2}) + \Theta(n) \\ T(1) = \Theta(1) \end{cases}$$

Essa ricade nel caso 2 del teorema principale, la cui soluzione è $T(n) = \Theta(n \log n)$.

Timsort

Nonostante il Merge Sort funzioni in tempo $O(n \log n)$ mentre l'Insertion Sort in $O(n^2)$, i fattori costanti sono tali che l'Insertion Sort è più veloce del Merge Sort per valori piccoli di n . Quindi, ha

senso usare l'Insertion Sort dentro il Merge Sort quando i sottoproblemi diventano sufficientemente piccoli.

L'algoritmo descritto nel testo può essere il seguente:

```
def Merge_Insertion (A,k, primo, ultimo, dim):
    if dim>k:
        medio =(primo+ultimo)//2
        Merge_Insertion (A,k, primo, medio, medio-primo+1)
        Merge_Insertion (A,k, medio+1, ultimo, ultimo-medio)
        Fondi(A, primo, medio, ultimo)
    else: InsertionSort(primo, ultimo)
```

L'equazione di ricorrenza associata a questa funzione è:

$$\begin{cases} T(n) = 2T(\frac{n}{2}) + \Theta(n) \\ T(k) = \Theta(k^2) \end{cases}$$

Che può essere risolta con il metodo iterativo che dopo i passi si presenta come:

$$T(n) = 2^i T(\frac{n}{2^i}) + \sum_{j=0}^{i-1} 2^j \Theta(\frac{n}{2^j}) \quad \frac{n}{2} \approx k \quad i = \log \frac{n}{k}$$

$$T(n) = 2^{\log \frac{n}{k}} \Theta(k^2) + \Theta(n) \sum_{j=0}^{\log \frac{n}{k} + 1} 1 = \Theta(\frac{n}{k} k^2) + \Theta(n) \log \frac{n}{k}$$

che è dell'ordine di $n \log n$ se $k = O(\log n)$.

5.5 Quicksort

L'algoritmo *quicksort* (ordinamento veloce) è un algoritmo che ha una caratteristica molto interessante: nonostante abbia un costo che nel caso peggiore è $O(n^2)$, nella pratica si rivela spesso la soluzione migliore per grandi valori di n perché: il suo tempo di esecuzione atteso è $\Theta(n \log n)$; i fattori costanti nascosti sono molto piccoli; permette l'ordinamento "in loco". Anche il quicksort sfrutta la tecnica algoritmica del divide et impera:

- **divide:** nella sequenza di n elementi viene selezionato un elemento detto *pivot*. La sequenza viene quindi divisa in due sotto-sequenze: la prima delle due contiene elementi minori o uguali del pivot, la seconda contiene elementi maggiori o uguali del pivot;
- **passo base:** la ricorsione procede fino a quando le sotto-sequenze sono costituite da un solo elemento;
- **impera:** le due sotto-sequenze vengono ordinate ricorsivamente;
- **combina:** non occorre alcuna operazione poiché la sequenza è già ordinata.

Lo pseudocodice dell'algoritmo quicksort è il seguente:

```
def Quick_sort (A, indice_primo, indice_ultimo):
    if (indice_primo < indice_ultimo):
        indice_medio = Partiziona (A, indice_primo, indice_ultimo)
        Quick_sort (A, indice_primo, indice_medio-1)
        Quick_sort (A, indice_medio+1, indice_ultimo)
```

Partiziona seleziona il valore dell'elemento più a destra e lo usa come pivot per partizionare la sequenza in tre regioni: alla fine del procedimento la regione di sinistra conterrà solo elementi minori o uguali al pivot, quella di destra conterrà solo elementi maggiori o uguali al pivot, e quella centrale, formata da un solo elemento, contenente il pivot.

```

def Partiziona (A, indice_primo, indice_ultimo):
    pivot = indice_ultimo // scelta arbitraria
    i = indice_primo - 1
    for j in range(indice_primo, indice_ultimo):
        if (A[j] <= A[pivot]):
            i += 1
            A[i], A[j] = A[j], A[i]
    A[i+1], A[indice_ultimo] = A[indice_ultimo], A[i+1]
    return i+1

```

Prima di analizzare il costo computazionale del Quicksort, vediamo il suo funzionamento su un piccolo esempio.

Esempio 8. Sia dato l'array $A = [2, 8, 7, 1, 3, 5, 6, 4]$; la funzione `Partiziona` modifica l'array in modo che esso divenga $A = [2, 1, 3, 4, 7, 5, 6, 8]$ e restituisce l'indice 3 (corrispondente al pivot con valore 4). A questo punto, la funzione `Quick_sort` viene richiamata ricorsivamente sui due sotto-array $[2, 1, 3]$ e $[7, 5, 6, 8]$.

Proseguendo le chiamate sui sotto-array che via via scaturiscono dai vari valori che vengono restituiti dalla funzione `Partiziona`, si giunge, infine, all'array ordinato.

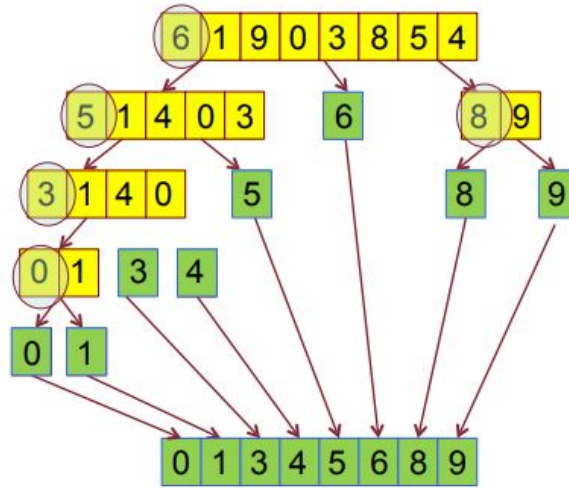


Figura 5.2: Quick sort

Per il costo computazionale, l'equazione di ricorrenza del quicksort va dunque espressa come segue:

$$\begin{cases} T(n) = T(k) + T(n-1-k) + \Theta(n) \\ T(1) = \Theta(1) \end{cases} \quad 0 \leq k \leq n-1$$

che non sappiamo come risolvere con alcuno dei metodi illustrati precedentemente. Possiamo però valutarla in casi specifici: nel caso peggiore, in quello migliore e in quello medio.

Caso peggiore

Il caso peggiore è quello in cui, ad ogni passo, la dimensione di uno dei due sotto-problemi da risolvere è nulla. In tale situazione l'equazione di ricorrenza è:

$$T(n) = T(n-1) + \Theta(n)$$

Applicando il metodo dell'albero: otteniamo un albero degenero¹. Il costo dei nodi decresce linearmente da n (nella radice) ad 1 (nell'unica foglia). Quindi il costo computazionale è:

$$T(n) = \Theta \sum_{i=1}^n i = \Theta(n^2)$$

¹Un albero binario si dice degenero se ha n nodi e altezza $n-1$, cioè se ogni nodo interno ha un solo figlio.

Caso migliore

Si può dimostrare che il caso caso migliore è quello in cui *Partiziona* ottiene due sotto-problemi il più bilanciati possibile, e in questo caso la ricorrenza diviene:

$$T(n) = 2T\left(\frac{n-1}{2}\right) + \Theta(n)$$

che, similmente a come già visto per il mergesort, ha soluzione $T(n) = \Theta(n \log n)$.

Caso medio

Valutiamo il costo computazionale del caso medio, nell'ipotesi che il valore del pivot suddivida con uguale probabilità $\frac{1}{n}$. Partendo da:

$$\begin{cases} T(n) = \frac{1}{n} [\sum_{k=0}^{n-1} (T(k) + T(n-1-k))] + \Theta(n) \\ T(0) = T(1) = \Theta(1) \end{cases}$$

Per ogni valore di $k = 0, 1, \dots, n-1$ il termine $T(i)$ compare due volte nella sommatoria, la prima quando $k = i$ e la seconda quando $k = n-1-i$. Valutiamo dunque il valore di:

$$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + \Theta(n)$$

Utilizziamo il metodo di sostituzione, e quindi eliminiamo innanzitutto la notazione asintotica:

$$T(n) = \frac{2}{n} \left[\sum_{k=0}^{n-1} T(k) \right] + an$$

Ipotizziamo ora la soluzione $T(n) \leq cn \log n$. E calcoliamo il caso base $T(2) = \frac{2}{2} + 2a = 2a \leq 2c$, risulta vera per $c \geq a$. Per il passo induttivo possiamo scrivere:

$$T(n) \leq \frac{2}{n} \left[\sum_{k=2}^{n-1} ck \log k \right] + an = \frac{2c}{n} \left[\sum_{k=2}^{n-1} k \log k \right] + an = \frac{2c}{n} \left[\sum_{k=1}^{n-1} k \log k \right] + an$$

Essendo $\sum_{i=1}^{n-1} i \log i$ una serie nota che risulta essere $\leq \frac{n^2}{n} \log n - \frac{n^2}{8}$, diciamo che:

$$\frac{2c}{n} \left[\sum_{k=1}^{n-1} k \log k \right] + an \leq \frac{2c}{n} \left(\frac{n^2}{2} \log n - \frac{n^2}{8} \right) + an = cn \log n - \frac{cn}{4} + an$$

Volendo $-\frac{cn}{4} + an \leq cn \log n$ minore di zero, ovvero per $c \geq 4a$ otteniamo:

$$cn \log n - \frac{cn}{4} + an \leq cn \log n$$

5.6 Heapsort

L'algoritmo *heapsort* è un algoritmo di ordinamento piuttosto complesso che esibisce ottime caratteristiche. Inoltre, questo algoritmo è l'occasione per iniziare a illustrare un concetto molto importante, quello di struttura dati. Sfrutta, infatti, una opportuna organizzazione dei dati che garantisce una o più specifiche proprietà, il cui mantenimento è essenziale per il corretto funzionamento dell'algoritmo.

Che cos'è l'heap?

L'heap (mucchio) è una struttura dati, nello specifico è un albero binario completo o quasi completo, ossia un albero binario in cui tutti i livelli sono pieni, tranne l'ultimo.

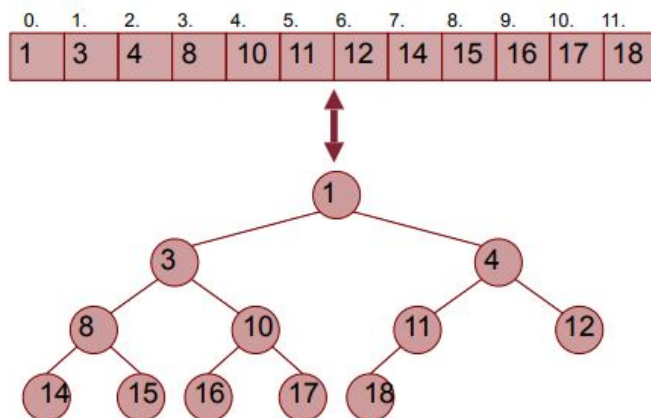
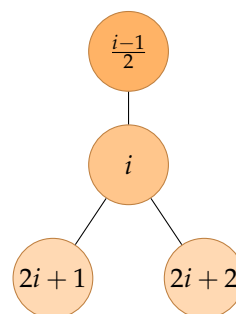


Figura 5.3: Esempio di heap

Il modo più naturale per memorizzare questa struttura dati è utilizzare un array. Ogni nodo dell'albero binario corrisponde a uno e un solo elemento dell'array, la radice dell'albero corrisponde ad $A[0]$. Il figlio sinistro del nodo che corrisponde all'elemento $A[i]$, se esiste, corrisponde all'elemento $A[2i + 1]$, mentre quello destro corrisponde all'elemento $A[2i + 2]$. Il padre del nodo dell'elemento $A[i]$ corrisponde all'elemento $A[\frac{i-1}{2}]$.

Le funzioni che implementa questa particolare struttura sono:

- La **creazione** dell'heap che avviene in tempo $O(n)$;
- L'**estrazione del minimo** che avviene in tempo $O(\log A)$ (A è la dimensione dell'heap\ array);
- L'**inserimento** di un valore che avviene in tempo $O(\log A)$ (A è la dimensione dell'heap\ array).



In particolare queste funzioni vengono gestite dai seguenti metodi:

```
import heapq
heapify(A)
x = heappop(A)
heappush(A, x)
```

Ora non sarà difficile capire l'idea dietro l'algoritmo dell'heapsort:

- si organizzano gli elementi in A da ordinare come heap minimo;
- si estraggono gli elementi dell'heap uno alla volta per inserirli in un nuovo vettore B inizialmente vuoto;
- si restituisce il vettore B .

Ad ogni estrazione si estrae da A il valore minimo quindi gli elementi finiranno nel nuovo vettore in ordine crescente.

```
import heapq
def Heap_Sort(A):
    heapq.heapify(A)
    B=[]
    while A:
        B.append(heapq.heappop(A))
    return B
```

La complessità temporale dell'algoritmo dovrebbe risultare $\Theta(n \log n)$, ma in questa specifica versione l'algoritmo non ordina in loco e lo spazio di lavoro è $O(n^2)$ a causa dell'utilizzo del vettore B .

Heappop()

I passaggi da seguire per l'implementazione della funzione `Heappop()` sono:

- salviamo in una variabile il minimo presente in $A[0]$;
- ricopiamo in $A[0]$ l'ultimo elemento $A[n-1]$ dell'heap e cancelliamo da A l'ultimo elemento;
- ci ritroviamo con un heap dove l'unico elemento fuori posto è quello alla radice e possiamo riaggiustare l'heap con una sola chiamata di `heapify` sull'elemento in posizione 0;
- restituiamo l'elemento.

```
def Heappop(A):
    x = A[0]
    A[0] = A[len(A)-1]
    A.pop()
    Heapify(A,0)
    return x
```

La complessità di questa funzione, per via dell'invocazione di `Heapify()`, risulta essere $O(\log n)$.

Heappush()

I passaggi da seguire per l'implementazione della funzione `Heappush()`:

- aggiungiamo l'elemento x all'ultimo posto dell'heap;
- fino a che non risulta maggiore del padre o raggiunge la radice facciamo risalire nell'heap.

```
def Heappush(A, x):
    A.append(x)
    i = len(A)-1
    while i>0 and A[i]<A[(i-1)//2]:
        A[i], A[(i-1)//2] = A[(i-1)//2], A[i]
        i = (i-1)//2
```

La complessità della procedura dipende dal numero di iterazioni del `while` e questo numero è $O(\log n)$.

Heapify()

L'algoritmo di `Heapify` si avvale di una funzione ausiliaria `Heapify1`, necessaria per il suo corretto funzionamento. La funzione `Heapify1` ha lo scopo di mantenere la proprietà di heap, sotto l'ipotesi che nell'albero su cui viene fatta lavorare sia garantita la proprietà di heap per entrambi i sotto-alberi (sinistro e destro) della radice. Di conseguenza l'unico nodo che può violare la proprietà di heap è la radice dell'albero, che può essere minore di uno o di entrambi i figli. La funzione opera sulla radice confrontandola coi suoi figli e, se necessario, la scambia col minore dei suoi figli. Dopo che lo scambio si verifica se la violazione si è trasferita sul figlio scambiato si ripete ricorsivamente l'operazione su tale nodo.

```
def Heapify1(A, i):
    n = len(A)
    L = 2*i+1; R = 2*i+2
    indice_min = i
    if L < len(A) and A[L] < A[indice_min]:
        indice_min = L
    if R < len(A) and A[R] < A[indice_min]:
        indice_min = R
    if indice_min != i:
        A[i], A[indice_min] = A[indice_min], A[i]
        Heapify1(A, indice_min)
```

Calcoliamo ora il costo computazionale di `Heapify1`. Dalle chiamate ricorsive otteniamo $T(n) = T(n') + \Theta(1)$ dove n è il numero di nodi del sotto-albero radicato in i e dove n' è il numero di nodi del sotto-albero di che ha più nodi. I sotto-alberi della radice non possono avere più di $\frac{2n}{3}$ nodi. Sia $n' = 2^h - 1$ il numero di nodi nel sotto-albero di sinistra.

$$n' = 2^{h+1} - 1 - \frac{2^h}{2} = \frac{3 * (2^h - 1) + 1}{2} = \frac{3n' + 1}{2} \quad n' = \frac{2n - 1}{3} < \frac{2n}{3}$$

Quindi l'equazione di ricorrenza diventa $T(n) \leq T(\frac{2n}{3}) + \Theta(1)$, che ha soluzione: $T(n) = O(\log n)$.

Possiamo ora passare all'analisi di `Heapify`. `Heapify` serve per trasformare qualunque vettore contenente elementi in uno heap, chiamando ripetutamente `Heapify1` sugli opportuni nodi dello heap. Inoltre poiché `Heapify1` presuppone che entrambi i sotto-alberi della radice siano heap, deve essere chiamata scorrendo l'albero per livelli dal basso verso l'alto (quindi, sul vettore, da destra a sinistra). Ogni foglia è già uno heap, quindi possiamo chiamare `Heapify` a partire dal nodo interno più a destra.

```
def Heapify(A):
    for i in range((len(A)-2)//2, -1, -1):
        Heapify1(A, i)
```

La funzione `Heapify` effettua n chiamate di `Heapify1`, che sappiamo avere ciascuna costo $O(\log n)$, quindi: $T(n) = O(n \log n)$.

Non resta che dimostrare che $T(n) = \Theta(n)$. Considerando un heap di altezza h . Nell'heap a livello i ci sono al più 2^i nodi, quindi il numero di nodi dell'heap che sono radice di un sotto-albero di altezza i sono $2^{h-1} = \frac{2^h}{2^i}$. Il tempo richiesto da `Heapify` applicata ad un nodo che è radice di un sotto-albero di altezza i è $O(i)$. Quindi possiamo scrivere:

$$T(n) \leq \sum_{i=1}^h 2^{h-i} O(i) = O(2^h \sum_{i=1}^h \frac{i}{2^i}) = O(2^h \sum_{i=1}^h (\frac{3}{4})^i)$$

perchè $\frac{i}{2^i} < (\frac{3}{4})^i$. Quindi otteniamo $T(n) = O(2^h) + O(1) = O(n)$.

Ordinamenti stabili

Un algoritmo di ordinamento si dice stabile se mantiene l'ordine relativo per gli elementi che hanno chiavi uguali. In altre parole, se due elementi hanno la stessa chiave, un algoritmo di ordinamento stabile garantirà che l'elemento che compare prima nella lista originale venga posizionato prima nella lista ordinata. Al contrario, un algoritmo di ordinamento non stabile può riordinare gli elementi con chiavi uguali in modo imprevedibile.

Ad esempio l'algoritmo di `SelectionSort` non è stabile. Per convincersene basta considerare cosa accade quando si ordina con questo metodo la lista di tre elementi $[2, 2, 1]$, si ottiene la lista ordinata $[1, 2, 2]$ ma l'ordinamento originario dei due elementi uguali risulta invertito. Degli algoritmi di ordinamento passati in rassegna non sono stabili gli algoritmi di `SelectionSort`, `QuickSort` ed `HeapSort`. Sono stabili gli algoritmi di `BubbleSort`, `InsertionSort`, `MergeSort` e `TimSort`.

Ordinamento in tempo lineare

È possibile ordinare n elementi in un tempo inferiore a $\Omega(n \log n)$? Il teorema che stabilisce una limitazione inferiore al costo degli algoritmi di ordinamento sembrerebbe dire di no; in realtà la risposta è sì, purchè l'algoritmo di ordinamento non sia basato sui confronti fra elementi poiché, in tal caso, come abbiamo visto nessun algoritmo può scendere sotto tale limite.

5.7 Counting Sort

Un esempio è il *counting sort* (algoritmo basato sul conteggio). Esso opera sotto l'assunzione che ciascuno degli n elementi da ordinare sia un intero di valore compreso in un intervallo $[0..k]$ ed ha

un costo di $\Theta(n + k)$. Se $k = O(n)$ allora l'algoritmo ordina n elementi in tempo lineare, cioè $\Theta(n)$. L'idea è quella di fare in modo che il valore di ogni elemento della sequenza determini direttamente la sua posizione nella sequenza ordinata. Questo semplice schema deve essere leggermente complicato per gestire l'eventualità che nella sequenza da ordinare vi siano più elementi con lo stesso valore o manchi un qualche valore. Oltre all'array A che contiene gli n elementi da ordinare, usiamo un array C di appoggio, contenente k celle.

```
def Counting_sort(A):
    k=max(A)
    n=len(A)
    C=[0]*(k+1)
    for j in range(n):
        C[A[j]]+= 1
    j=0
    for i in range(k):
        while (C[i] > 0):
            A[j]=i
            j+= 1
            C[i]-= 1
```

La prima istruzione calcola il massimo dell'array A e costa, quindi, $\Theta(n)$. L'inizializzazione a 0 dell'array C costa $\Theta(k)$. Il primo ciclo conta, per ciascun indice di C , il numero di elementi di A aventi tale valore. Il secondo ciclo scandisce l'array C e, al suo interno, per ogni valore di i , un ciclo `while` scrive $C[i]$ copie di i sull'array A . Il numero complessivo di iterazioni del ciclo `while` è pari alla somma dei valori contenuti in C , che è esattamente n , quindi il costo computazionale è:

$$\Theta(n) + \Theta(k) + \Theta(n) + \Theta(k + n) = \Theta(\max(k, n)) = \Theta(n) \text{ se } k = \Theta(n).$$

Segue un piccolo esempio del funzionamento dell'algoritmo di Counting Sort. Sia dato l'array $A = [2, 5, 1, 2, 3, 3, 5, 0, 3, 1]$. In questo caso $n = 10$ e $k = 5$. Dopo l'esecuzione del ciclo `for j in range(n)`, l'array C contiene i seguenti valori: $C = [1, 2, 2, 3, 0, 2]$ ed, ovviamente, risulta che la somma dei valori contenuti in C è proprio pari ad n . A questo punto, è facile ricostruire l'array A ordinato, scrivendo dapprima 3 occorrenze del valore 1, poi 2 occorrenze del valore 2, e così via, fino alla fine dell'array C : $A = [0, 1, 1, 2, 2, 3, 3, 3, 5, 5]$.

5.8 Bucket sort

Un altro algoritmo interessante, che ha costo computazionale medio lineare, è il *bucket sort*. Come il counting sort, il bucket sort è efficiente perché fa delle ipotesi sull'input. In particolare, assume che gli n valori dati in input da ordinare siano equamente distribuiti in un intervallo $[1..k]$, e non c'è alcuna ipotesi su k .

L'idea del bucket sort consiste nel dividere l'intervallo $[1..k]$ in n sotto-intervalli di uguale ampiezza $\frac{k}{n}$, detti bucket. A causa dell'ipotesi di equi-distribuzione, non ci si aspetta che molti valori dell'input cadano in ciascun intervallo, anzi in media essi saranno in numero costante. Il bucket sort introduce un array ausiliario di lunghezza n di array, ciascuno delle quali conterrà tutti gli elementi dati in input che cadono in un certo sotto-intervallo; più in dettaglio, un bucket i conterrà tutti i valori compresi tra $\frac{(i-k)}{n}$.

```
def Bucket_sort(A,k):
    M=max(A)
    k=min(k,M+1,len(A))
    B=[[ ] for _ in range(k)]
    for x in A:
        B[int (x*k/(M+1))].append(x)
    for i in range(k):
        B[i].sort()
    R=[ ]
    for i in range(k):
        R.extend(B[i])
    return R
```

Per quanto riguarda il costo computazionale dell'algoritmo di bucket sort, è chiaro che esso dipende fortemente dalla lunghezza delle liste $B[i]$. Poiché, come abbiamo già evidenziato, tale lunghezza è mediamente costante, è facile vedere che il costo computazionale medio di questo algoritmo è lineare. Invece, nel caso peggiore, esso sarà quadratico.

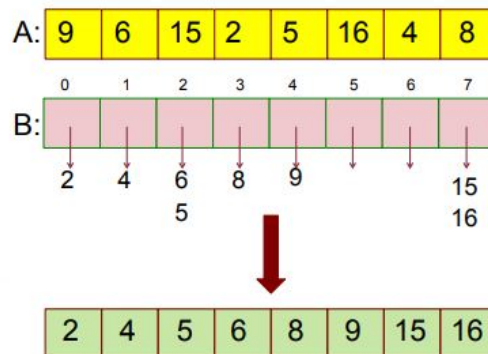


Figura 5.4: Bucket sort

Esercizi svolti

Esercizio 10 (Problema della ricerca). Sia dato un vettore di interi A e due valori a e b con $a \leq b$, il problema è quello di sapere quanti elementi di A sono compresi nell'intervallo chiuso $[a, b]$.

1. Si progetti un algoritmo per risolvere tale problema su qualsiasi vettore A ;
2. Si progetti un algoritmo più efficiente del precedente assumendo che A sia già ordinato e che contenga solo valori distinti.

Per il primo punto scorro il vettore e conto gli elementi inclusi nell'intervallo $[a, b]$.

```
def Conta_in_Intervallo(A, a, b):  
    cont = 0  
    for i in range(len(A)):  
        if a <= A[i] <= b:  
            cont += 1  
    return cont
```

Costo dell'algoritmo è $\Theta(n)$.

Per il secondo punto cerco a e b nel vettore con la ricerca binaria. Serve una piccola modifica all'algoritmo di ricerca binaria. Basta aggiungere `return m`. In questo modo per il valore m restituito possono accadere tre cose:

- $A[m] = v$. In questo caso v è in A ed m è la sua posizione;
- $A[m] < v$. In questo caso v non è in A ed è m la posizione dell'elemento minore di v più a destra presente in A ;
- $A[m] > v$. In questo caso v non è in A ed è m la posizione dell'elemento maggiore di v più a sinistra presente in A .

```
def Conta_in_Intervallo_ordinato(A, a, b):  
    ind_a = ricerca_binaria_modificata(A, a)  
    if A[ind_a] < a: ind_a += 1  
    ind_b = ricerca_binaria_modificata(A, b)  
    if A[ind_b] > b: ind_b -= 1  
    return ind_b - ind_a + 1
```

Siamo così riusciti a risolvere il problema in tempo $\Theta(\log n)$.

Esercizio 11 (Equazioni ricorsive). Calcolare la complessità della seguente equazione ricorsiva:

$$\begin{cases} T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + \Theta(n) \\ T(1) = \Theta(1) \end{cases}$$

In questo tipo di problemi vogliamo ottenere una forma del tipo:

$$T_1(n) \leq T(n) \leq T_2(n) \quad T_1(n) = \Omega \quad T_2(n) = O$$

Scriviamo:

$$T_1(n) = 2T(\frac{n}{3}) + \Theta(n)$$

applicando il metodo principale otteniamo $n = \Omega(n^{\log_3 2 + \epsilon})$ con risultato $\Theta(n)$.

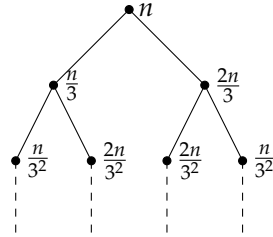
Scriviamo, invece:

$$T_2(n) = 2\left(\frac{2n}{3}\right) + \Theta(n)$$

che con il metodo principale ci rende $n = O(n^{\log_{\frac{3}{2}} 2 - \epsilon})$ con risultato $\Theta(n^{\log_{\frac{3}{2}} 2})$.

$$\Omega(n) \leq T(n) \leq O(n^{\log_{\frac{3}{2}} 2})$$

Notiamo che con questo metodo non riusciamo ad ottenere il valore effettivo della complessità asintotica, per tanto proviamo con il metodo dell'albero.



Sommando i costi in ogni fila di nodi otteniamo sempre $\Theta(n)$. Ma va aggiunto che i rami hanno altezze diverse:

Ramo più corto: $\frac{n}{3^i} \approx 1 \quad n \approx 3^i \quad i \approx \log_3 n$;

Ramo più lungo: $\frac{2n}{3^i} \approx 1 \quad n \approx \frac{3^i}{2} \quad i \approx \log_{\frac{3}{2}} n$.

Ottenendo così come risultato:

$$\Theta(n) \log_3 n \leq T(n) \leq \Theta(n) \log_{\frac{3}{2}} n \quad \Theta(n \log n) \leq T(n) \leq \Theta(n \log n)$$

$$T(n) = \Theta(n \log n)$$

Parte III

Strutture dati

Capitolo 6

Strutture dati fondamentali

Una struttura dati è un particolare tipo caratterizzato da un modo sistematico di organizzare i dati. Mette a disposizione un insieme di operatori che permettono di memorizzare e manipolare *insiemi dinamici*, cioè insiemi i cui elementi possono variare nel tempo in funzione delle operazioni compiute dall'algoritmo che li utilizza.

Come vedremo più approfonditamente in seguito, una struttura dati può essere *lineare o non lineare* (a seconda se esista una sequenzializzazione, e se possa essere pensata con un inizio ed una fine), *statica o dinamica* (a seconda che possa variare la sua dimensione nel tempo), *omogenea o disomogenea* (rispetto ai contenuti).

Insiemi dinamici

Gli elementi dell'insieme dinamico (spesso chiamati anche *nodi*, *record*, o *oggetti*) possono a loro volta essere piuttosto complessi e contenere ciascuno più di un dato "elementare". In tal caso è abbastanza comune che essi contengano: una *chiave*, utilizzata per distinguere un elemento da un altro, nell'ambito delle operazioni di manipolazione dell'insieme dinamico; *ulteriori dati*, che sono relativi all'elemento stesso ma non sono direttamente utilizzati nelle operazioni di manipolazione. Le tipiche operazioni che si compiono su un insieme dinamico si dividono in due categorie: operazioni di interrogazione (*Search*, *Minimum*, *Maximum*, *Predecessor*, *Successor*) e operazioni di modifica (*Insert*, *Delete*).

Le differenti strutture dati che vedremo, se da un lato hanno in comune la capacità di memorizzare insiemi dinamici, dall'altro differiscono anche profondamente fra loro per le proprietà che le caratterizzano. Sono proprio le proprietà della struttura in questione ad essere l'elemento determinante nella scelta da effettuare quando si deve progettare un algoritmo per risolvere un problema.

Un'ultima considerazione a proposito delle strutture dati è poiché sono destinate alla gestione di insiemi dinamici, è piuttosto comune una loro implementazione mediante l'allocazione dinamica della memoria, per contenerne gli elementi, e l'uso di puntatori, per accedere agli stessi. Alternativamente, possono essere implementate per mezzo di array purché il numero massimo degli elementi possa essere stimato in anticipo.

Prima di studiare alcune strutture dati, da quelle più semplici (*lista*, *coda* e *pila*) a quelle più complesse (*albero*, *grafo*), ne riprendiamo una che già abbiamo incontrato: l'*array*.

6.1 Array

Innanzitutto, osserviamo che un *array* (sia esso ordinato o no) ha le seguenti caratteristiche: *memorizza elementi omogenei*, cioè dello stesso tipo; è *statico*, cioè ha una lunghezza che deve essere definita a priori; è una struttura ad *accesso casuale*, cioè possiamo accedere ad ogni suo elemento in un tempo costante che è indipendente dalla sua posizione.

Cerchiamo ora di capire quale sia il costo computazionale di alcune delle operazioni sopra citate quando vengano implementate su vettori semplici o ordinati.

Struttura dati	Search	Min/Max	Pre/Succ	Insert	Delete
Array qualunque	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$O(1)$
Array ordinato	$O(\log n)$	$\Theta(1)$	$\Theta(1)$	$O(n)$	$O(n)$

Tabella 6.1: Costo computazionale delle operazioni negli array

Già da questo semplice confronto si può dedurre come non abbia senso dire che una struttura è migliore di un'altra: le strutture dati possono essere più o meno adatte ad un certo algoritmo e sta al buon progettista disegnare un algoritmo efficiente usando la struttura dati più adatta.

6.2 Liste

Un *puntatore* è una variabile di tipo particolare che contiene l'indirizzo in memoria di un'altra variabile. I puntatori sono necessari per costruire strutture dati dinamiche e per passare i parametri per riferimento.

La *lista* è una struttura dati nella quale gli elementi sono organizzati in successione. Le proprietà specifiche delle liste sono le seguenti: l'accesso avviene sempre ad una estremità della lista, per mezzo di un'apposita informazione di accesso (puntatore alla testa della lista); è permesso solo un accesso sequenziale agli elementi: non è possibile l'accesso diretto.

A differenza dell'array, la lista può crescere di dimensioni e la successione degli elementi è implementata mediante un collegamento esplicito da un elemento ad un altro. Il valore speciale None rappresenta "nessun puntatore" (viene raffigurato con il carattere \).



Figura 6.1: Lista

Lista puntata semplice

Ogni elemento di lista è un record a due campi: il campo *key* contiene l'informazione vera e propria; il campo *next* contiene il puntatore che consente l'accesso all'elemento successivo; nel caso dell'ultimo elemento della lista contiene un apposito valore None (visualizzato col simbolo \).

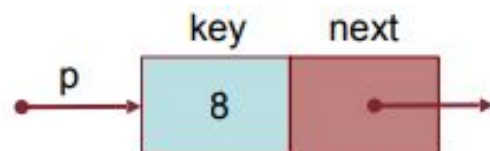
$p.key$ indica il contenuto del campo *key* dell'elemento puntato da p . Mentre $p.next$ indica il contenuto del campo *next* dell'elemento puntato da p , ossia l'indirizzo dell'elemento successivo (o None se esso è l'ultimo elemento).

Ad esempio in Python possiamo definire la classe `nodo` come segue:

```
class Nodo:
    def __init__(self, key=None, next=None):
        self.key=key
        self.next=next
```

La funzione `Crea` prende in input un array A , genera una lista semplice contenente un nodo per ogni elemento dell'array e termina restituendo la testa alla lista creata. Il costo computazionale della creazione è $\Theta(n)$.

```
def crea(A):
    if A == []: return None
    p = Nodo(A[0])
    q = p
    for i in range(1, len(A)):
        q.next = Nodo(A[i])
```




```

    q = q.next
    return p

```

La funzione `Stampa` prende in input il puntatore `p` alla testa di una lista semplice e stampa i contenuti di ciascun nodo della lista. Il costo computazionale della stampa è $\Theta(n)$.

```

def stampa(p):
    while p:
        print(p.key)
        p = p.next

```

La funzione `Ricerca`, dato il puntatore `p` alla testa di una lista semplice e un valore `x`, restituisce il puntatore al primo nodo della lista contenente il valore `x`, il puntatore restituito sarà `None` se nella lista un tale nodo non è presente. Il costo computazionale della ricerca è $O(n)$.

```

def ricerca(p, x):
    q = p
    while q != None and q.key != x:
        q = q.next
    return q

```

La funzione `Inserisci`, dato il puntatore `p` alla testa di una lista semplice ed un intero ad un nodo, inserisce un nodo con quel valore in testa alla lista e restituisce la nuova testa della lista. Il costo computazionale dell'inserimento in testa è $\Theta(1)$.

```

def Inserisci_dopo_q(p, x):
    q = Nodo(x, p)
    return q

```

La funzione `Cancella`, dato il puntatore `p` alla testa di una lista ed il puntatore `d` ad un nodo della lista lo cancella e restituisce la testa della lista (eventualmente modificata). Il costo computazionale della cancellazione è $O(n)$.

```

def cancella(p, d):
    if p == d: return p.next
    q = p
    while q.next != d:
        q = q.next
    q.next = d.next
    return p

```

Liste puntata doppia

Alcune complicazioni riscontrate nella lista semplice (ad esempio il costo computazionale nella cancellazione) possono essere risolti organizzando la struttura dati in modo che da ogni suo elemento si possa accedere sia all'elemento che lo segue che a quello che lo precede nella lista, quando essi esistono. Tale struttura dati si chiama *lista puntata doppia*.

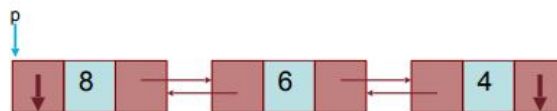


Figura 6.2: Lista puntata doppia

Ad esempio in Python possiamo definire la classe dei nodi di queste liste come:

```

class NodoD:
    def __init__(self, key=None, prev=None, next=None):
        self.key = key
        self.next = next
        self.prev = prev

```

Le operazioni di inserimento e ricerca sono analoghe alle precedenti (con le ovvie modifiche necessarie alla corretta gestione del campo `prev`). Se voglio “eliminare” dalla lista l’elemento puntato da `p`:

```
...
p.prev.next = p.next
p.next.prev = p.prev
...
```

Il costo computazionale è $\Theta(1)$.

Riassumiamo la situazione nella seguente tabella:

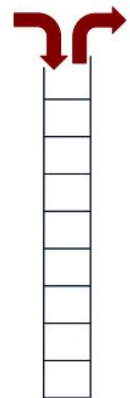
Struttura dati	Search	Min/Max	Pre	Insert	Delete
Lista semplice	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$O(n)$
Lista doppia	$O(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$

Tabella 6.2: Costo computazionale delle operazioni nelle liste

6.3 Pile

La *pila* è una struttura dati che esibisce un comportamento *LIFO* (Last In First Out). In altre parole, la pila ha la proprietà che gli elementi vengono prelevati dalla pila nell’ordine inverso rispetto a quello col quale vi sono stati inseriti.

Su una pila sono definite solo due operazioni: l’inserimento (che di norma viene chiamata *Push*) e l’estrazione (che di norma viene chiamata *Pop*). La particolarità che garantisce la proprietà LIFO è che le operazioni Push e Pop operano sulla stessa estremità della pila (attraverso il puntatore *top*).



Implementazione con liste puntate

La funzione Push riceve il puntatore alla cima della pila ed il valore da inserire, lo inserisce in un nodo in cima alla lista puntata e restituisce la nuova cima.

```
def push(top, x):
    top = Nodo(x, top)
    return top
```

La funzione Pop prende il puntatore alla cima della pila e restituisce il valore del nodo in cima alla pila e la nuova cima della pila.

```
def pop(top):
    if top == None:
        print('Errore_pila_vuota')
        return None, None
    return top.key, top.next
```

Implementazione con liste

```
def push(Pila, x):
    Pila.append(x)

def pop(Pila):
    if Pila == []: return None
    return Pila.pop()
```

Per entrambe le implementazioni (liste o liste puntate) il costo computazionale di Push e Pop, è $\Theta(1)$.

6.4 Code

La *coda* è una struttura dati che esibisce un comportamento *FIFO* (First In First Out). In altre parole, la coda ha la seguente proprietà: gli elementi vengono prelevati dalla coda esattamente nello stesso ordine col quale vi sono stati inseriti.

Su una coda sono definite solo due operazioni: l'inserimento (che di norma viene chiamata *Enqueue*) e l'estrazione (che di norma viene chiamata *Dequeue*). Viceversa, di solito non si scandiscono gli elementi di una coda né si eliminano elementi con mezzi diversi dalla *Dequeue*. La particolarità che garantisce la proprietà FIFO è che l'operazione *Enqueue* opera su una estremità della coda (*tail*) e la *Dequeue* opera sull'altra estremità (*head*):

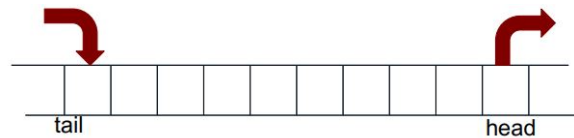


Figura 6.3: Rappresentazione di una coda

Implementazione con liste puntate

La funzione *Enqueue* prende in input i puntatori alla testa e alla coda ed il valore da inserire e dopo aver inserito il nodo col nuovo valore in coda restituisce i puntatori alla testa ed alla coda.

```
def inserisci(testa , coda , x):
    e = Nodo(x)
    if testa == None: return e, e
    coda.next = e
    return testa , e
```

La funzione *Dequeue* prende in input i puntatori alla testa e alla coda e dopo aver estratto il valore del nodo in testa lo restituisce con i puntatori alla testa ed alla coda.

```
def estrai(testa , coda):
    if testa == None:
        print('Errore: _coda_vuota')
        return None, None, None
    if testa == coda:
        return testa.key, None, None
    return testa.key, testa.next, coda
```

Implementazione con liste

```
def inserisci(Coda, testa , x):
    Coda.append(x)
    if testa == None: return 0
    return testa

def estrai(Coda, testa):
    if testa == len(Coda):
        print('Errore _coda_vuota')
        return None, testa
    return Coda[testa] , testa+1
```

Per entrambe le implementazioni (liste o liste puntate) il costo computazionale di Push e Pop, è $\Theta(1)$.

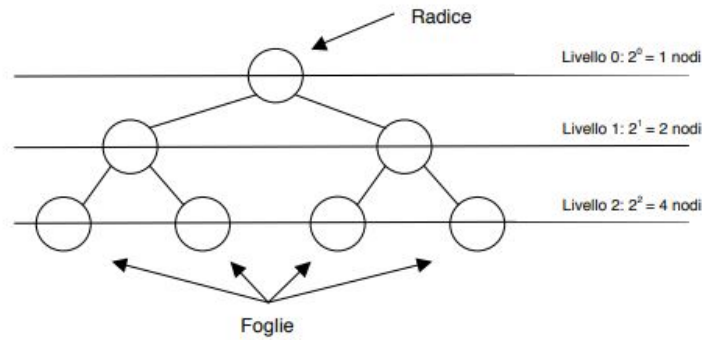


Figura 6.5: Albero binario

6.5 Alberi

Un *albero* è una struttura dati estremamente versatile, utile per modellare una grande quantità di situazioni reali e progettare le relative soluzioni algoritmiche.

Alberi radicati

In un *albero radicato*: i nodi sono organizzati in livelli, numerati in ordine crescente allontanandosi dalla radice (di norma la radice è posta a livello zero); l'altezza di un albero radicato è la lunghezza del più lungo cammino dalla radice ad una foglia; un albero di altezza h contiene $h+1$ livelli, di norma numerati da 0 ad h .

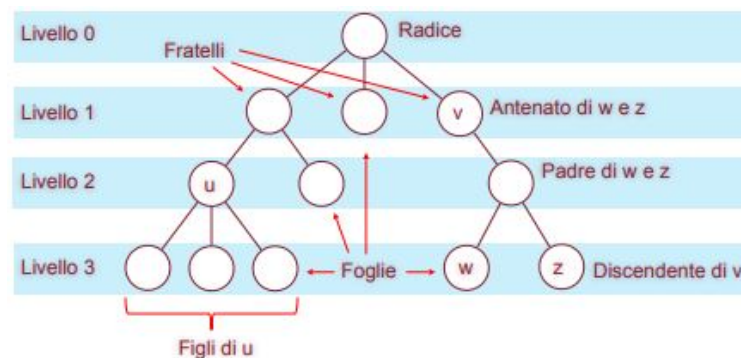


Figura 6.4: Albero radicato

Un albero radicato si dice *ordinato* se attribuiamo un qualche ordine ai figli di ciascun nodo, nel senso che se un nodo ha k figli, allora vi è un figlio che viene considerato primo, uno che viene considerato secondo, ..., uno che viene considerato k -esimo.

Una particolare sottoclasse di alberi radicati e ordinati è quella degli alberi binari.

Alberi binari

Gli *alberi binari* hanno la particolarità che ogni nodo ha al più due figli. Poiché sono alberi ordinati, i due figli di ciascun nodo si distinguono in *figlio sinistro* e *figlio destro*. Un albero binario nel quale tutti i livelli contengono il massimo numero possibile di nodi è chiamato *albero binario completo*. Se invece tutti i livelli tranne l'ultimo contengono il massimo numero possibile di nodi, l'albero è chiamato *albero binario quasi completo*.

Il numero delle foglie di un albero binario è 2^h ; il numero dei nodi interni è $\sum_{i=0}^{h-1} 2^i = \frac{2^h - 1}{2 - 1} = 2^h - 1$; il numero totale dei nodi è $2^h + 2^h - 1 = 2^{h+1} - 1$. In ultima analisi l'altezza h di un albero binario completo è dato dal numero dei nodi $n = 2^{h+1} - 1$ da cui segue che $\log_2(n + 1) = h + 1$ e

quindi:

$$h = \log_2(n+1) - 1 = \log_2 \frac{n+1}{2} \quad h = \Theta(\log n)$$

In generale per un albero binario si ha $\Omega(\log n) = h \leq n - 1$.

Rappresentazione di un albero binario

Il modo più naturale di rappresentare e gestire gli alberi binari è per mezzo dei puntatori. Ogni singolo nodo è costituito da un record contenente: *key*: le opportune informazioni pertinenti al nodo stesso; *left*: il puntatore al figlio sinistro (oppure *None* se il nodo non ha figlio sinistro); *right*: il puntatore al figlio destro (oppure *None* se il nodo non ha figlio destro).

Si accede all'albero per mezzo del puntatore alla radice.

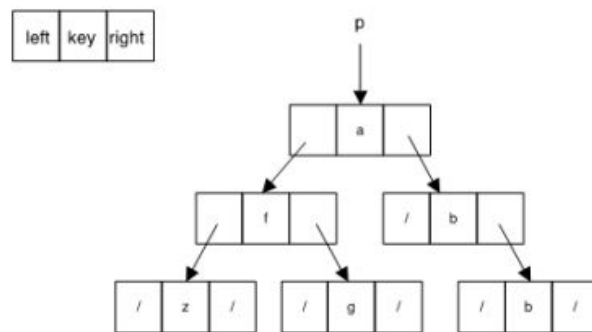


Figura 6.6: Albero binario con puntatori

Questo tipo di memorizzazione ha tutti i vantaggi e l'elasticità delle strutture dinamiche basate sui puntatori (si possono inserire nuovi nodi, spostare dei nodi ecc.). In una struttura di questo tipo il codice per la creazione di ogni nodo dell'albero è il seguente:

```
class NodoAB:
    def __init__(self, struct, key=None, fs=None, fd=None):
        struct.key = key
        struct.fs = fs
        struct.fd = fd
```

Ora abbiamo tutto il necessario per creare un albero binario random (dove n rappresenta il numero di nodi e m il valore delle chiavi di ogni singolo nodo in un intervallo $[0, m]$):

```
import random
def crea(n, m):
    if n == 0: return None
    r = NodoAB(random.randint(0, m))
    s = random.randint(0, n-1)
    r.fs = crea(s, m)
    r.fd = crea(n-s-1, m)
    return r
```

Visita in un albero binario

Un'operazione basilare sugli alberi, che spesso è il presupposto per poter risolvere problemi su di essi, è l'accesso a tutti i suoi nodi, uno dopo l'altro, al fine di poter effettuare una specifica operazione (che dipende ovviamente dal problema posto) su ciascun nodo. Abbiamo visto come tale operazione sulle liste sia una semplice iterazione, ma sugli alberi la situazione è più complessa dato che la loro struttura è ben più articolata.

L'accesso progressivo a tutti i nodi di un albero si chiama *visita dell'albero*. Facendo riferimento all'ordine col quale si accede ai nodi dell'albero, è evidente che esiste più di una possibilità. Nel caso specifico degli alberi binari, nei quali i figli di ogni nodo (e quindi i sottoalberi) sono al massimo due,

e volendo comportarsi nello stesso modo su tutti i nodi, le possibili decisioni in merito a questa scelta danno luogo a tre diverse visite:

- *preorder*: il nodo è visitato prima di proseguire la visita nei suoi sottoalberi;
- *inorder*: il nodo è visitato dopo la visita del sottoalbero sinistro e prima di quella del sottoalbero destro;
- *postorder*: il nodo è visitato dopo entrambe le visite dei sottoalberi.

Con le nozioni apprese possiamo risolvere dei semplici ma utili problemi sugli alberi.

1. Dato il puntatore alla radice stampare un albero binario di n nodi:

```
def stampa(r):  
    if r == None: return  
    print(r.key)  
    stampa(r.fs)  
    stampa(r.fd)
```

2. Dato il puntatore alla radice calcolare il numero di nodi di un albero:

```
def conta(r):  
    if r == None: return 0  
    return 1 + conta(r.fs) + conta(r.fd)
```

3. Dato il puntatore alla radice calcolare l'altezza dell'albero:

```
def altezza(r):  
    if r == None: return 0  
    a = altezza(r.fs)  
    b = altezza(r.fd)  
    return max(a,b) + 1
```

Capitolo 7

Dizionari

Un dizionario è una struttura dati che permette di gestire un insieme dinamico di dati tramite queste tre sole operazioni:

- **insert**: si inserisce un elemento;
- **search**: si ricerca un elemento;
- **delete**: si elimina un elemento.

7.1 Tabelle Hash

Una *tabella hash* (in inglese hash table) è una struttura di dati efficiente per implementare dizionari. Sotto ragionevoli assunzioni, le tabelle hash implementano le tre operazioni di inserimento, cancellazione e ricerca con complessità media costante $\Theta(1)$.

Per implementare una tabella hash bisogna dimensionare la tabella in base al numero di elementi attesi (che indicheremo con m) ed utilizzare una speciale funzione (funzione hash) per indicizzare la tabella.

Funzione Hash

Una *funzione hash* è una funzione che data una chiave x restituisce la posizione della tabella in cui l'elemento con chiave x viene memorizzato: $h(x) \in \{0, 1, \dots, m-1\}$. La dimensione m della tabella può non coincidere con l'insieme degli elementi n , anzi in generale $m \ll n$.

L'idea è quella di definire una funzione d'accesso che permetta di ottenere la posizione di un elemento data la sua chiave in questo modo riduciamo lo spazio necessario per memorizzare la tabella ma perdiamo la corrispondenza tra chiavi e posizioni in tabella. Inoltre le tabelle hash possono soffrire del fenomeno delle collisioni, dato che il numero di posizioni è molto minore a quello degli elementi da inserire, capiterà che la funzione restituisca lo stesso valore per entrambe le chiavi, che quindi andrebbero memorizzate nella stessa posizione della tabella.

Una buona funzione hash dovrà essere in grado di distribuire uniformemente le chiavi nello spazio degli indici. La situazione ideale è quella in cui ciascuna delle m posizioni della tabella è scelta con la stessa probabilità.

Un metodo semplicissimo per definire una funzione hash è dello del modulo: $h(k) = k \bmod m$. Nella maggior parte dei casi questo metodo dà buoni risultati ma vi sono situazioni in cui si può essere particolarmente sfortunati. Pertanto, nei successivi paragrafi, illustreremo i due metodi più diffusi per la gestione delle collisioni (indirizzamenti chiusi e aperti).

Indirizzamento chiuso

In un *indirizzamento chiuso* ad ogni cella della tabella hash si fa corrispondere una lista (solitamente una lista concatenata) detta *lista di trabocco*. In questo modo un elemento che collide viene aggiunto alla lista corrispondente all'indice ottenuto.

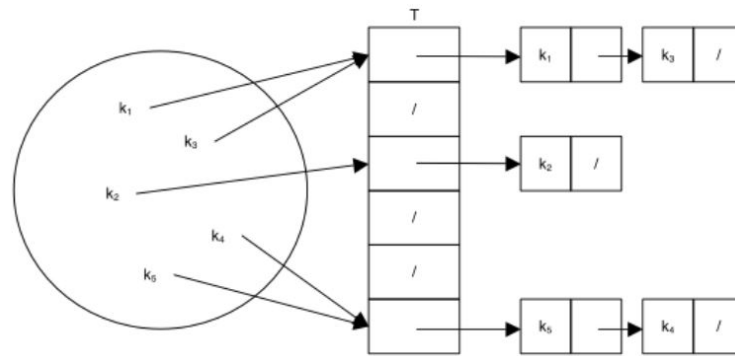


Figura 7.1: Indirizzamento chiuso con liste di trabocco

Analizziamo ora le principali operazioni da compiere:

- L'*inserimento* avviene in $\Theta(1)$, essendo una lista a puntatori si può sempre inserire in tempo costante in testa alla lista;
- Nella *ricerca* il caso pessimo si verifica quando tutti gli elementi memorizzati nella tabella hash mappano alla medesima posizione generando una complessità dell'ordine $O(n)$. Nel caso medio, invece, (quando la funzione hash gode di uniformità semplice) è $O(1 + \frac{n}{m}) = O(1 + \alpha)$ dove con α indichiamo il fattore di carico della tabella;
- Per la *cancellazione* infine dipende dall'implementazione delle liste di trabocco e valgono, pertanto, tutte le osservazioni fatte per il costo dell'operazione di cancellazione nelle liste.

Indirizzamento aperto

L'*indirizzamento aperto* prevede di inserire tutti gli elementi direttamente nella tabella, senza far uso di strutture dati aggiuntive. Essa è applicabile quando: $n \leq m$ quindi il fattore di carico ($\frac{n}{m}$) è $\alpha < 1$.

Supponiamo di voler inserire una chiave k e che la sua posizione "naturale" $h(k)$ sia già occupata. L'indirizzamento aperto consiste nell'occupare un'altra cella vuota, anche se essa potrebbe spettare di diritto ad un'altra chiave. Se la posizione iniziale relativa alla chiave k è occupata, si scandisce la tabella fino a trovare una posizione libera nella quale l'elemento con chiave k può essere memorizzato.

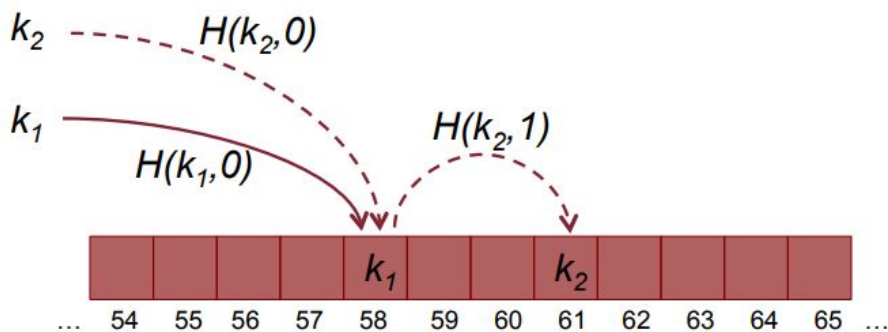


Figura 7.2: Indirizzamento aperto

Anche in questo caso analizziamo il costo delle operazioni:

- Per l'*inserimento*, quando si incontra una collisione, non si fa altro che utilizzare l'indice successivo a quello che collide sino a che non si trovi una casella libera. Oppure si utilizza un'ulteriore funzione hash in modo tale che, se le due funzioni sono ben progettate, è estremamente improbabile che due chiavi k_1 e k_2 producano una collisione su entrambe le funzioni hash;
- L'operazione di *ricerca* è simile a quella di inserimento. Se durante la scansione delle celle viene trovata una cella che contiene la chiave l'operazione restituisce l'elemento trovato. Se invece si arriva ad una cella vuota o si è scandita senza successo tutta la tabella si restituisce None.

- Per la *cancellazione* la prima cosa che verrebbe in mente è quella di eliminare con la cella dell'array che contiene la chiave. Così facendo la ricerca non sarebbe più corretta infatti potrebbe interrompersi prima di trovare l'elemento cercato per via di un "buco" creato dalla cancellazione (in pratica potrebbe accadere che un elemento presente in tabella non venga trovato). Quindi marchiamo le celle che hanno subito cancellazione con un valore speciale *canc.*

7.2 Alberi binari di ricerca

Gli *alberi binari di ricerca* (ABR) sono alberi binari nei quali vengono mantenute le seguenti proprietà:

- ogni nodo dell'albero contiene una chiave;
- il valore della chiave contenuta in ogni nodo è maggiore al valore della chiave contenuta in ciascun nodo del suo sottalbero sinistro (se esso esiste);
- il valore della chiave contenuta in ogni nodo è minore al valore della chiave contenuta in ciascun nodo del suo sottalbero destro (se esso esiste).

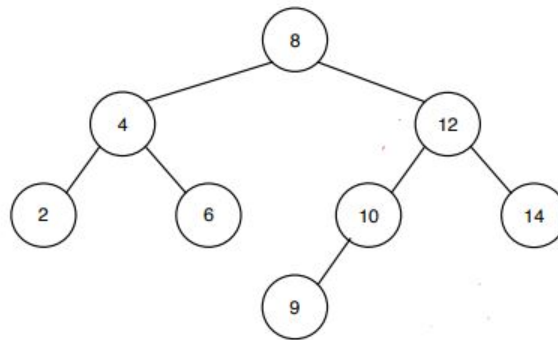


Figura 7.3: Albero binario di ricerca

Per inciso in un ABR il minimo è sempre nel nodo più a sinistra, il massimo in quello più a destra. Si noti che il nodo più a sinistra non necessariamente è una foglia; analoga considerazione per il nodo più a destra.

Ricerca su un albero binario di ricerca

Il procedimento di ricerca su un ABR è concettualmente simile alla ricerca binaria ed è molto semplice: si esegue una discesa dalla radice che viene guidata dai valori memorizzati nei nodi che si incontrano lungo il cammino. Quando si trova in un nodo il valore ricercato si termina con successo restituendo il puntatore al nodo. Se invece si arriva ad una foglia che non lo contiene si termina senza successo restituendo `Null`.

```

def ricercaABR(p, x):
    if p == None or p.key == x:
        return p
    if p.key < x:
        return ricercaABR(p.right, x)
    return ricercaABR(p.left, x)
  
```

Per quanto riguarda il costo computazionale della funzione di ricerca, intuitivamente, dato che la ricerca percorre un singolo cammino dalla radice ad una foglia, ci aspettiamo che esso sia legato all'altezza h dell'albero.

$$T(h) = O(h)$$

Come mai non riusciamo a garantire un costo logaritmico per la ricerca come per gli alberi binari? La ragione risiede nel fatto che l'ABR non include fra le sue proprietà alcuna relazione con la sua altezza, infatti sono alberi binari di ricerca anche alberi con un solo ramo lungo h .

Inserimento in un albero binario di ricerca

Anche il procedimento di inserimento su un ABR è molto semplice: si esegue una discesa che, come nel caso della ricerca, viene guidata dai valori memorizzati nei nodi che si incontrano lungo il cammino. Quando si arriva al punto di voler proseguire la discesa verso un puntatore vuoto (Null) allora, in quella posizione, si aggiunge un nuovo nodo contenente il valore da inserire.

In realtà il metodo più efficiente consiste nell'aggiunta di un nuovo campo ai nodi dell'albero in cui viene memorizzato il puntatore al padre (parent).

Massimo e minimo in un albero binario di ricerca

Il *minimo* (*massimo*) si trova nel nodo più a sinistra (destra), quindi per trovarlo si scende sempre a sinistra (destra) a partire dalla radice. Ci si ferma quando si arriva a un nodo che non ha figlio sinistro (destro): quel nodo contiene il minimo (massimo).

Predecessore e successore in un albero binario di ricerca

Per *predecessore* di k si intende il nodo dell'albero contenente la chiave che precederebbe immediatamente k se le chiavi fossero ordinate. Notiamo come se il nodo che contiene k ha il sotto-albero sinistro, il suo predecessore è il massimo di tale sotto-albero. Mentre se il nodo che contiene k non ha sotto-albero sinistro vuol dire che esso è il nodo più a sinistra di un certo sotto-albero. Per trovare il suo predecessore bisogna risalire alla radice di quel sotto-albero, il che significa salire a destra finché è possibile e una volta giunti nella radice del sotto-albero, si risale (con un singolo passo di salita a sinistra) a suo padre che è il predecessore di k .

Una situazione perfettamente simmetrica esiste per il problema di trovare il *successore* di un nodo, dove per successore si intende il nodo che contiene la chiave che seguirebbe immediatamente k se le chiavi fossero ordinate.

Cancellazione in un albero binario di ricerca

Il problema di eliminare una chiave contenuta in un ABR è il più complicato, poichè si possono verificare vari casi in base al numero di figli del nodo che si vuole cancellare:

- se il nodo è una foglia lo si elimina, ponendo a NULL l'opportuno campo nel nodo padre $\Theta(1)$;
- se il nodo ha un solo figlio si collegano direttamente fra loro suo padre e il suo unico figlio, indipendentemente che sia destro o sinistro $\Theta(1)$;
- se il nodo ha entrambi i figli lo si sostituisce con il massimo contenuto nei suoi sotto-alberi. A sua volta il massimo va eliminato dalla sua posizione originale. Si noti che, per trovare il massimo si cercherà sempre nel sotto-albero destro e per cancellarlo si userà sicuramente il caso 1 o il caso 2, infatti il massimo non può avere il figlio destro perchè altrimenti non sarebbe lui il massimo (il figlio destro avrebbe un valore maggiore di quello del padre) $O(h)$;

7.3 Alberi di ricerca bilanciati

Come abbiamo visto, un ABR di altezza h contenente n nodi supporta le operazioni fondamentali dei dizionari in tempo $O(h)$, ma purtroppo non si può escludere che h sia $O(n)$, con conseguente degrado delle prestazioni. Viceversa, tutte le operazioni restano efficienti se si riesce a garantire che l'altezza dell'albero sia piccola, in particolare sia $O(\log n)$.

Per raggiungere tale scopo esistono varie tecniche, dette di *bilanciamento*, tutte basate sull'idea di riorganizzare la struttura dell'albero se essa, a seguito di un'operazione di inserimento o di eliminazione di un nodo, viola determinati requisiti. In particolare, il requisito da controllare è che, per ciascun nodo dell'albero, l'altezza dei suoi due sotto-alberi non sia "troppo differente". Ciò che rende non banali queste tecniche è che si vuole aggiungere agli alberi di ricerca una proprietà (il bilanciamento) senza peggiorare il costo computazionale delle operazioni.

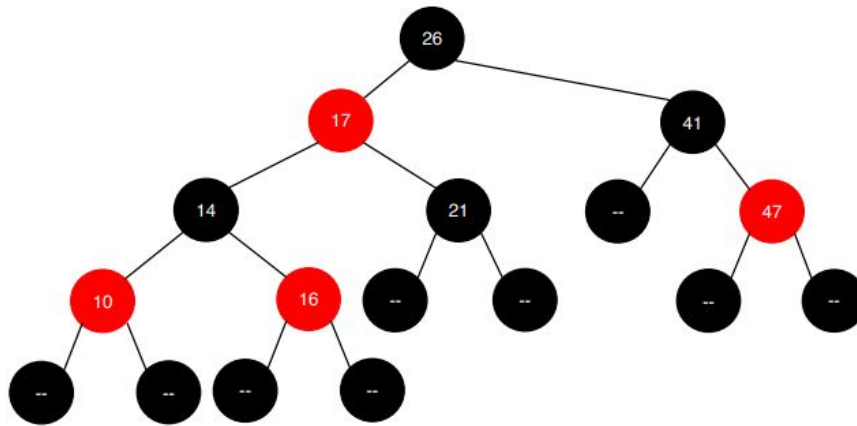


Figura 7.4: Albero red-black

Alberi red-black

Illustreremo ora una di tali tecniche, quella degli RB-alberi (*alberi Red-Black*).

Un RB-albero è un ABR i cui nodi hanno un campo aggiuntivo, il colore, che può essere solo rosso o nero. Alla struttura si aggiungono foglie fittizie (che non contengono chiavi) per fare in modo che tutti i nodi “veri” dell’albero abbiano esattamente due figli.

Un RB-albero è un ABR che soddisfa le seguenti proprietà aggiuntive:

1. ciascun nodo è rosso o nero;
2. ciascuna foglia fittizia è nera;
3. se un nodo è rosso i suoi figli sono entrambi neri;
4. ogni cammino da un nodo a ciascuna delle foglie del suo sotto-albero contiene lo stesso numero di nodi neri.

Col termine b-altezza di x (black height di x , $bh(x)$) si indica il numero di nodi neri sui cammini dal nodo x alle foglie sue discendenti. La b-altezza di un RB-albero è la b-altezza della sua radice.

Dalla combinazione delle proprietà 3 e 4 discende un fatto interessante: nessun cammino dalla radice ad una foglia può essere lungo più del doppio di un cammino dalla radice ad una qualunque altra. Questo perché il numero di nodi neri è il medesimo lungo tutti i cammini (proprietà 4) e il numero di nodi rossi lungo un cammino non può essere maggiore del numero di nodi neri lungo lo stesso cammino (proprietà 3). Quindi un cammino che contiene il massimo numero possibile di nodi rossi non può essere lungo più del doppio di un cammino composto solo di nodi neri. Questa proprietà è sufficiente a dimostrare che l’altezza di un RB-albero di n nodi è $O(\log n)$.

Questa proprietà ci garantisce che le operazioni di ricerca di una chiave, del massimo o del minimo, del predecessore o del successore, siano tutte eseguite con un costo computazionale $O(\log n)$. Discorso a parte va fatto invece per gli inserimenti e le cancellazioni. Infatti, l’esigenza di mantenere le proprietà di RB-albero implica che, dopo un inserimento o una cancellazione, la struttura dell’albero possa dover essere riaggiustata, in termini di: colori assegnati ai nodi o struttura dei puntatori (ossia, collocazione dei nodi nell’albero).

A tal fine sono definite apposite operazioni, dette *rotazioni*, che permettono di ripristinare in tempo $O(\log n)$ le proprietà del RB-albero dopo un inserimento o un’eliminazione. Le rotazioni, che possono essere destre o sinistre, sono operazioni locali che non modificano l’ordinamento delle chiavi secondo la visita in-ordine, e possono essere schematizzate come riportato nella figura seguente:

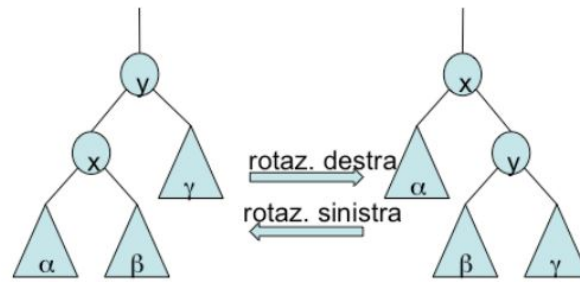


Figura 7.5: Rotazioni destre e sinistre di un albero red-black

Possiamo osservare che effettuare ciascuna rotazione implica un costo costante poiché coinvolge un numero costante di puntatori, essendo un'operazione locale.

```
def RBA_rotaz_sinistra(p, a):
    b = a.right
    a.right = b.left
    if b.left != None:
        b.left.parent = a
    b.left = a
    b.parent = a.parent
    if a == p:
        p = b
    elif a == a.parent.left:
        a.parent.left = b
    else:
        a.parent.right = b
    a.parent = b
    return p
```

Lo codice della rotazione a destra è analogo.

Possiamo ora descrivere l'algoritmo di inserimento di una chiave in un albero red-black. Si utilizza l'algoritmo di inserimento in un ABR, colorando il nodo sempre di rosso. L'unica regola che può essere infranta è la 3 (entrambi i figli di un nodo rosso sono neri), infatti il nuovo nodo (rosso) potrebbe essere inserito come figlio di un nodo rosso.

Dobbiamo dunque procedere ad aggiustare la struttura solo in questo caso. Si presentano 3 possibili eventualità da gestire in modo diverso:

1. Il nodo inserito è figlio di un nodo rosso e lo zio è rosso;
2. Il nodo inserito è figlio destro di un nodo rosso e lo zio è nero;
3. Il nodo inserito è figlio sinistro di un nodo rosso e lo zio è nero.

Per il primo caso si cambiano i colori di alcuni nodi: detto x il nuovo nodo appena inserito, si colorano di nero il padre e lo zio di x , di rosso il nonno di x . A questo punto, o la violazione è stata eliminata o è stata portata più in alto, e si ripresenta di nuovo. Per il secondo caso si effettua una rotazione a sinistra imperniata sul padre di x , e questo conduce sempre al terzo caso. Per il terzo caso si effettua una rotazione a destra e eventualmente si cambiano i colori dei figli, il che garantisce sempre la soluzione della violazione.

Ne consegue che, durante un inserimento, è possibile entrare più volte nel caso 1, eventualmente nel caso 2, e si conclude sempre con il caso 3. Poiché ogni volta che si presenta un nuovo caso da risolvere, la violazione si è spostata più in alto, avremo al più $O(\log n)$ iterazioni di costo costante, provando così che in un albero red-black si può inserire una nuova chiave in tempo $O(\log n)$.

Lo stesso vale per la cancellazione.

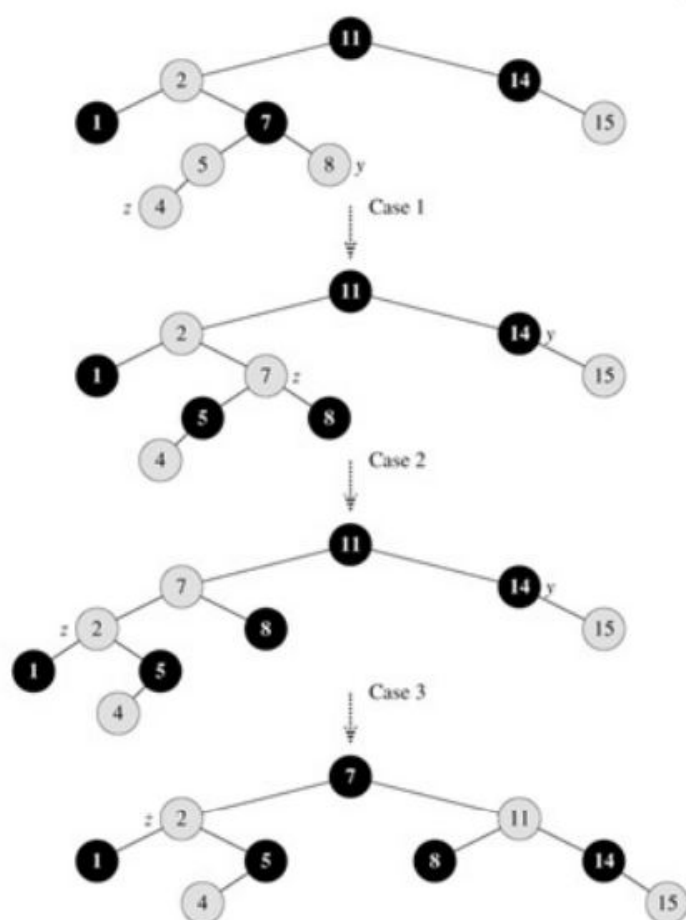


Figura 7.6: Casi in uso per gli alberi red-black

Esercizi svolti

Esercizio 12 (Liste).

- Dato il puntatore di una lista, scrivere una funzione che in tempo $\Theta(n)$ restituisce la lista invertita. Il programma non deve generare nuovi nodi ma lavorare sui nodi della lista originaria.

```
def inverti(p):  
    q = None  
    while p != None:  
        s = p  
        p = p.next  
        s.next = q  
        q = s  
    return q
```

- Dato il puntatore di una lista ordinata ed un intero x scrivere una funzione che in tempo $\Theta(n)$ inserisce un nodo con chiave x nella lista in modo che questa risulti ancora ordinata in modo crescente e restituisce il puntatore alla lista.

```
def inserimentoOrdinato(p, x):  
    s = Nodo(x)  
    if p == None or x > p.key:  
        s.next = p  
        return s  
    q = p  
    while q.next != None and q.next.key < x:  
        q = q.next  
    s.next = q.next  
    q.next = s  
    return p
```